Green for the Ambulance: How the Digital-Twin Dataset Is Generated

Methodology and equations. Technical companion to the ITI5145 proposal *Green for the Ambulance: A Digital-Twin Dataset for Learning Emergency-Vehicle Priority at Signalised Intersections* (business scenario: healthcare, emergency medical services)

Group members: [Name, Student ID]; [Name, Student ID]; [Name, Student ID]. ITI5145 Foundations of Data Science, Monash University, Term 4 2026

Generator version traffic_twin_v1.2, configuration hash ecd36a4b3cad; measured numbers from the 120-episode run, compiled 2026-10-03

- 0. Purpose, reading guide and notation
  - 0.1 How this document maps to the proposal and to the report
- 1. Pipeline overview
- 2. Arena geometry
- 3. Vehicle model
  - 3.1 Driver parameters: one latent aggressiveness
  - 3.2 Car-following: the Intelligent Driver Model
  - 3.3 Ballistic update with an exact stop
  - 3.4 Desired speed, speeding and turn anticipation
  - 3.5 Two lanes per direction without lane changing
  - 3.6 Who is my leader?
  - 3.7 Stop or commit at red onset
  - 3.8 Per-vehicle bookkeeping
- 4. Demand and routing
  - 4.1 Arrivals: a Poisson process by coin flips
  - 4.2 Destinations and routes
  - 4.3 Free-flow time and delay
- 5. The signal plant: the safety shield
- 6. Controllers and label functions
- 7. The ambulance extension (healthcare scenario)
- 8. Dataset schema
  - 8.1 Tables and grain
  - 8.2 The ML table, `signal_ticks` (80 columns)
  - 8.3 Seeds, streams and determinism
  - 8.4 Scenario mixture, splits and preprocessing
- 9. Demand calibration from the measured saturation flow
- 10. Validation experiments and results
- 11. Step-by-step: generating the dataset
- 12. Material for the Assignment 3 report, under the brief's section names
  - 12.0 Front matter the report must carry
  - 12.1 Characterising and Analysing Data (a): data sources and the four V's
  - 12.2 Characterising and Analysing Data (b): platforms, software and tools, now and if the project expands

    - 12.3 Characterising and Analysing Data (c): data analysis techniques and statistical methods, rationale and expected outcomes
    - 12.4 Characterising and Analysing Data (d): demonstration in R
    - 12.5 Results of the pilot models
    - 12.6 Standard for Data Science Process, Data Governance and Management
    - 12.7 Use of generative AI (statement for the report)
- 13. Limitations and future work
- References
- Appendix A. Worked numeric examples
- Appendix B. Implementation notes: where the generator refines the written design
- Appendix C. Data dictionary of `signal_ticks`

0. Purpose, reading guide and notation

The proposal says what the project will do and why: build a simulation-based digital twin of a four-intersection, left-hand-traffic grid, generate open tabular data on ambulance passages through it, and use that data to train a supervised signal controller that cuts ambulance travel time while keeping general-traffic delay within a stated bound. This document says exactly *how* the data behind that promise are produced: the road network, the physics of every vehicle, the way demand is generated, how the traffic lights decide, how the tables are logged, how the demand level is calibrated, how the simulator is validated, and how the resulting CSV files feed the models and the Assignment 3 report. Every equation the generator uses is written out, followed by a one-sentence plain-English reading for readers without a traffic-engineering background.

Vehicle-level records of ambulance passages cannot be collected safely on live roads, so the project generates them with a validated simulator; synthetic data from a validated generator are the accepted substitute when real collection is unsafe or impossible (Jordon et al., 2022). In the vocabulary of Kušić et al. (2023) a transport digital twin is a model, a live data link and a synchronisation loop; version 1 delivers the model layer and is built so that the other two can be added later without changing the data schema (Section 1); a simulated version of both has since been built on top of it (Section 13). The simulator is purpose-made rather than SUMO so that two-colour signals, per-driver parameter distributions and left-hand two-lane geometry are first-class (proposal Section 2.3); SUMO remains the planned external validation backend (Sections 12.2 and 13).

0.1 How this document maps to the proposal and to the report

Proposal goal (proposal Section 1.3)	Specified in
1. The twin: a 2×2 grid, left-hand traffic, two lanes per direction, red/green signals with internal all-red clearance, Intelligent Driver Model drivers, Poisson arrivals, origin–destination routing, an ambulance vehicle type	Sections 2–7
2. The dataset: at least 100 reproducible episodes as CSV tables with a data dictionary and a data card	Sections 8, 9 and 11
3. The model: a gradient-boosted model that predicts, per candidate phase, the general queue 30 s ahead and the ambulance delay, deployed as a priority-aware controller behind a safety shield	Sections 5, 6, 7 and 12.3
4. The evaluation: paired random seeds against fixed-time, max-pressure and naive full preemption; ambulance travel time primary, general delay and recovery time secondary; confidence intervals over at least eight seeds	Sections 7, 10 and 12.3

The Assignment 3 report must present the project under the five section names of the unit brief within 2,500 words (reused proposal text counts; references do not). Sections 1 to 3 of the report revise the proposal. Section 12 of this document is written so that the report's remaining sections can be lifted from it directly:

Report section (named as in the brief)	Material in this document	Indicative words
4. Characterising and Analysing Data: data sources and the four V's; platforms, software and tools; analysis techniques and statistical methods; demonstration in R	12.1, 12.2, 12.3 and 12.4, with 12.5 supplying the results	about 1,100
5. Standard for Data Science Process, Data Governance and Management	12.6	about 250
Front matter and the generative-AI statement	12.0 and 12.7	outside the five sections

Configuration and provenance. Every constant below is a field of one configuration object. The full configuration is saved as JSON, together with a short hash of it, beside every generated dataset, and the hash and the generator version are stamped on every episode row. Numbers marked *measured* were read from the 120-episode run and the validation run; they are regenerated by the build step of this document, not typed in.

Figures. Figures A1–A7 are static diagrams numbered by subject, not by order of first appearance; Figures V1–V4 are outputs of the validation run (Section 10). Figures A1–A7 carry their number and title inside the image, and the caption below each paraphrases it; Figures V1–V4 are reproduced as the validation run writes them, with panel titles only, and take their number from the caption.

Units. SI throughout: metres (m), seconds (s), m/s, m/s². Traffic rates in vehicles per hour (veh/h) and vehicles per hour per lane (veh/h/lane). The coordinate origin is the centre of the south-west junction, x east, y north.

Time axis of an episode. $t = 0$ is the start of the warm-up. Warm-up $[0, 300)$ s, main period $[300, 3900)$ s, cool-down $[3900, 4200]$ s with spawning switched off. KPIs and labels use only the main period.

Symbol	Meaning	Default
Δt	physics time step	0.5 s
s, v, a	position of the front bumper along its edge, speed, acceleration	—
v_0	a driver's desired (free-flow) speed	sampled
$a_{\max}, b$	maximum acceleration, comfortable deceleration	sampled
T	desired time headway	sampled
s_0, l_{veh}	standstill gap, vehicle length	2.0 m, 4.5 m
b_{emerg}, b_{stop}	physical braking limit, stop-or-commit deceleration	7.0, 3.0 m/s²
v_{limit}, v_{turn}	speed limit, speed cap inside a turning movement	13.9, 5.0 m/s
$L_{block}, L_{stub}, w_{lane}$	junction spacing, stub length, lane width	200, 150, 3.5 m
w_{box}	side of the square junction box	14 m
$\lambda_e(t)$	arrival rate at boundary node e	calibrated
g, r_{all}, C	displayed green, all-red, fixed-time cycle	20, 4, 96 s
g_{eff}, l	effective green, lost time per phase	18, 6 s
s_{sat}, c, x	saturation flow, approach capacity, volume-to-capacity ratio	measured
n_a, q_a	vehicles on approach a; of which queued ($v < 2$ m/s)	logged
p_a	max-pressure value of approach a	computed
`ev_*`	emergency-vehicle (ambulance) columns and random stream	Section 7

Approaches are named by the arm a vehicle arrives from: N, E, S, W. Movements are T (through), NS (near-side turn, the one that does not cross oncoming traffic: a left turn under left-hand traffic) and FS (far-side turn, which crosses oncoming traffic: a right turn under left-hand traffic). *Junction* is this document's word for the proposal's *intersection*. Junctions are $J(i, j)$ with i east, j north; `junction_id` $= j \cdot \text{grid}_x + i$.

1. Pipeline overview

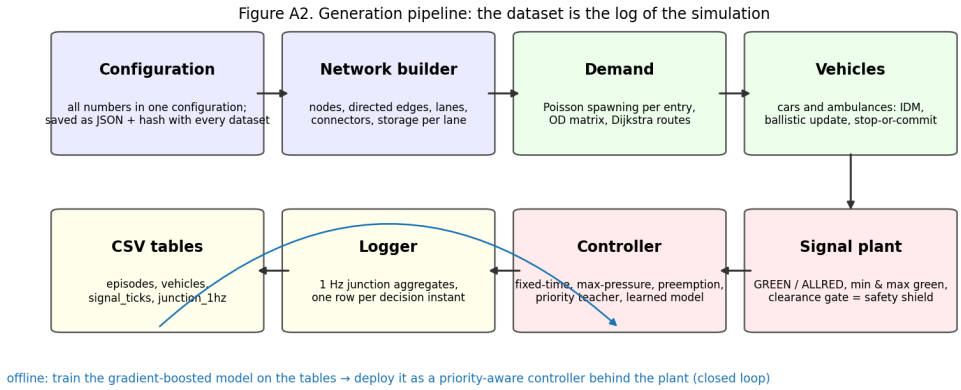


Figure A2. Generation pipeline: the dataset is the log of the simulation.

The dataset cannot be written before the simulation exists: it *is* the log of the simulation. The proposal's business model rests on it: the dataset is the product, serving as training data for the controller (Section 12.3), as evidence for an investment decision through the paired-seed comparison (Sections 8.3 and 12.3) and as a reusable benchmark (Sections 8 and 12.6). One episode is one run of the world with a given seed and scenario. The order of operations inside every half-second step determines what each logged row "sees":

1. **1 Hz log.** On whole seconds, write the per-junction aggregates (`junction_1hz`) and, for showcase episodes, every vehicle's state.
2. **Signals.** Handle green onsets whose clearance gate has opened; at each junction's decision instant build the feature row, ask the controller for a request, pass the request through the plant, evaluate stop-or-commit on the approach that lost its green, and log one `signal_ticks` row.
3. **Spawning.** For each boundary node, draw whether a new car (and, if enabled, an ambulance) requests entry; insert the head of the node's entry queue if there is room.
4. **Accelerations.** For every vehicle, find its leader and compute the Intelligent Driver Model acceleration.
5. **Movement.** Advance vehicles (connectors first, then edges), handle stop-line crossings, box entry and despawning at the boundary.
6. **Overlap guard, bookkeeping, plant timers, gridlock check.**

At the end of the episode a post-pass computes the labels (`y_mp_action` from each row's own columns, `y_q_total_t30` from the 1 Hz log) and the episode KPIs. The same feature-building function produces the row the controller acts on **and** the row that is logged, so the training schema and the serving schema are identical by construction. That is what makes the model layer ready for the live data link the proposal plans: a state estimate built from real detectors and ambulance GPS, fed into the same function, yields a row the trained model can consume.

2. Arena geometry

Figure A1. The '#' arena: 4 junctions, 8 boundary nodes, 2 lanes per direction

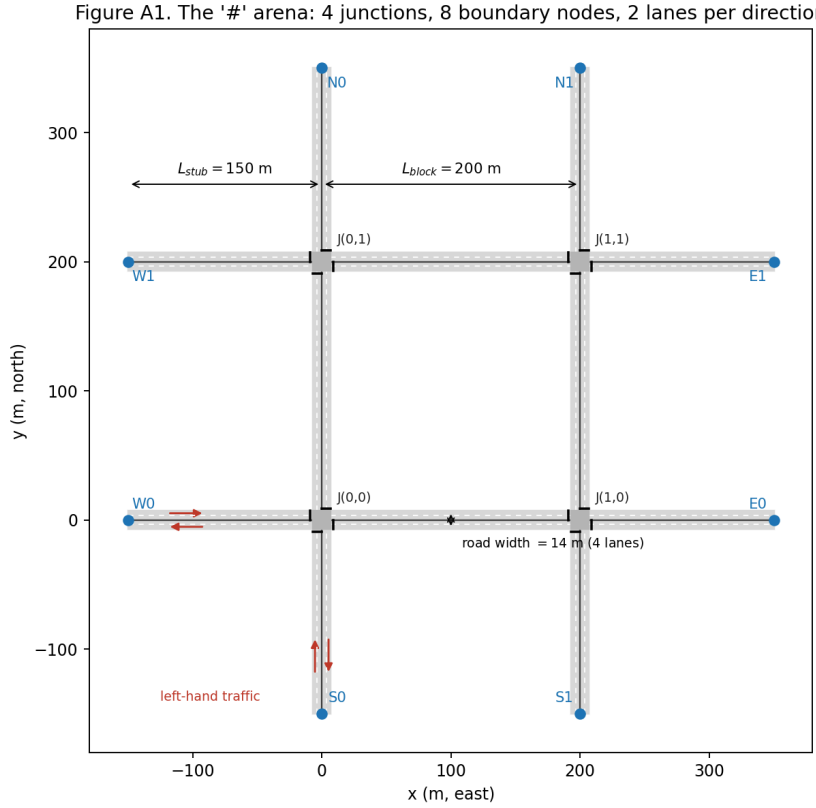


Figure A1. The "#" arena to scale: 4 junctions, 8 boundary nodes, 2 lanes per direction.

Two horizontal and two vertical two-way roads cross to form a 2×2 grid of four-way junctions; the roads continue past the grid to eight boundary nodes that are both sources and sinks. Left-hand traffic with two lanes per direction is the proposal's Jakarta geometry. The network builder accepts any $\text{grid}_x \times \text{grid}_y$ (the single-junction 1×1 case is used for validation).

Road, box and edge lengths. With $n_{lanes} = 2$ lanes per direction,

$$w_{road} = 2\, n_{lanes}\, w_{lane} = 14 \text{ m}, \qquad w_{box} = w_{road},$$
$$L_{int} = L_{block} - w_{box} = 186 \text{ m}, \qquad L_{stub}^{edge} = L_{stub} - \frac{1}{2} w_{box} = 143 \text{ m}.$$

Directed edges run from the edge of one box to the edge of the next (or to a boundary node). The stop line sits 2 m upstream of the box: $s_{stop} = L_e - 2$. Inside a box, each of the 12 movements (4 approaches $\times$ 3 movements) is a *connector* of length w_{box}; turning radii are drawing-only.

Lane geometry by heading rotation. Let $\mathbf{h} = (h_x, h_y)$ be the edge's unit heading and $\mathbf{n} = (-h_y, h_x)$ its left normal. Lane k ($k = 0$ kerb lane, $k = n_{lanes} - 1$ median lane) is offset from the road axis by

$$\mathbf{o}_k = \sigma\, \mathbf{n}\, w_{lane}\left(n_{lanes} - k - \frac{1}{2}\right), \qquad \sigma = +1 \text{ (left-hand traffic)}, \; -1 \text{ (right-hand)},$$

so the kerb lane is 5.25 m and the median lane 1.75 m to the left of the axis. A vehicle at distance s along the edge is drawn at $(x, y) = \mathbf{P}_{start} + \mathbf{h}\, s + \mathbf{o}_k$. *Reading:* under left-hand traffic every car drives on the left of the centre line as seen from the driver, and the generator never hard-codes which side that is.

Exit arm of a movement. With the rotation matrix $R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$,

$$\mathbf{h}_{out}^{T} = \mathbf{h}, \qquad \mathbf{h}_{out}^{NS} = R(+90^\circ\sigma)\,\mathbf{h}, \qquad \mathbf{h}_{out}^{FS} = R(-90^\circ\sigma)\,\mathbf{h}.$$

The unit test that pins this down: a southbound vehicle, $\mathbf{h} = (0, -1)$, under left-hand traffic turns near-side to $(1, 0)$, the **east** arm. No U-turns, no turns on red.

Storage. The number of vehicles a lane can hold bumper to bumper is

$$S_{lane}(L) = \left\lfloor \frac{L}{l_{veh} + s_0} \right\rfloor = \left\lfloor \frac{L}{6.5} \right\rfloor,$$

giving 28 per internal lane and 22 per stub lane (56 and 44 per edge). Queue features are normalised by the storage of their own edge. The static table `network_edges.csv` lists, for each of the 24 edges: id, end nodes, heading, length, lanes, storage, whether it is a stub, the approach name and junction at its destination, and its endpoint coordinates. Node and edge ids follow the notation of Section 0: junction $J(i, j)$ is written `Jij`; a boundary node is `S` or `N` followed by the index i of its vertical road, or `W` or `E` followed by the index j of its horizontal road; a directed edge id is `<from>_<to>`, so `S0_J00` is the stub entering the south-west junction from the south and `J00_J01` the northbound internal edge from $J(0, 0)$ to $J(0, 1)$.

3. Vehicle model

3.1 Driver parameters: one latent aggressiveness

Each vehicle draws its parameters once, at the moment it requests entry (so that vehicles are identical across controllers for the same seed). A latent aggressiveness $\alpha \sim \text{Beta}(2, 2)$ on $[0, 1]$ shifts every trait in a correlated way, and a lognormal factor adds individual scatter:

$$\theta = \bar{\theta}\left(1 + c_\theta\,(\alpha - 0.5)\right) e^{\sigma_\theta Z}, \qquad Z \sim \mathcal{N}(0, 1),$$

followed by clipping to a plausible range.

Trait θ	Mean $\bar{\theta}$	Slope c_θ	Scatter σ_θ	Clip
desired speed v_0	v_{limit} = 13.9 m/s	+0.20	0.08	$[0.85, 1.15]\, v_{limit}$
max acceleration $a_{\max}$	1.5 m/s²	+0.50	0.10	[0.8, 2.5]
comfortable deceleration b	2.0 m/s²	+0.30	0.10	[1.2, 3.0]
time headway T	1.2 s	−0.40	0.10	[0.8, 1.8]

Reading: an aggressive driver (large α) wants to go faster, accelerates and brakes harder, and follows more closely. α is stored in `vehicles.csv` as a hidden variable that can later serve as a regression target. The mean headway of 1.2 s was chosen so that the measured saturation flow lands in the 1600–1900 veh/h/lane range that the design adopted as its acceptance band for a through lane (Section 10, Experiment 1).

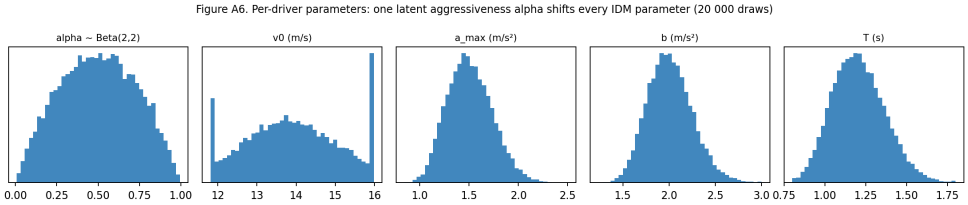


Figure A6. Per-driver parameters: one latent aggressiveness shifts every IDM parameter (20,000 draws).

3.2 Car-following: the Intelligent Driver Model

Every vehicle's acceleration is the Intelligent Driver Model (IDM) of Treiber et al. (2000), the standard car-following model of microscopic simulation:

$$a = a_{\max}\left[1 - \left(\frac{v}{v_0^{loc}}\right)^4 - \left(\frac{s^*(v, \Delta v)}{g}\right)^2\right],$$

$$s^*(v, \Delta v) = s_0 + \max\left(0,\; v\,T + \frac{v\,\Delta v}{2\sqrt{a_{\max}\,b}}\right), \qquad \Delta v = v - v_{lead},$$

where g is the bumper-to-bumper gap to the leader and s^* the gap the driver wants. *Reading:* accelerate towards your preferred speed, ease off as you approach it, and brake harder the closer you are to the car

ahead relative to the gap you want; the wanted gap is a standing buffer plus about T seconds of travel, plus more when you are closing in. On a free road (no leader within 100 m) the last term vanishes.

The IDM term is never clamped inside the formula (clamping destroys its collision-free property). After computing a: if $a < -3$ m/s² while the vehicle is moving, a *hard brake* is counted (a standing vehicle cannot brake); then $a \leftarrow \max(a, -b_{emerg})$ with $b_{emerg} = 7$ m/s² as the physical limit. A non-positive gap (two vehicles abreast in different lanes or connectors that aim for the same box slot) is never fed to the IDM; the yielding vehicle instead drops back at its own comfortable b (Section 3.5).

Figure A4. Intelligent Driver Model (a=1.5, b=2.0, T=1.2 s, s0=2 m, v0=13.9 m/s)

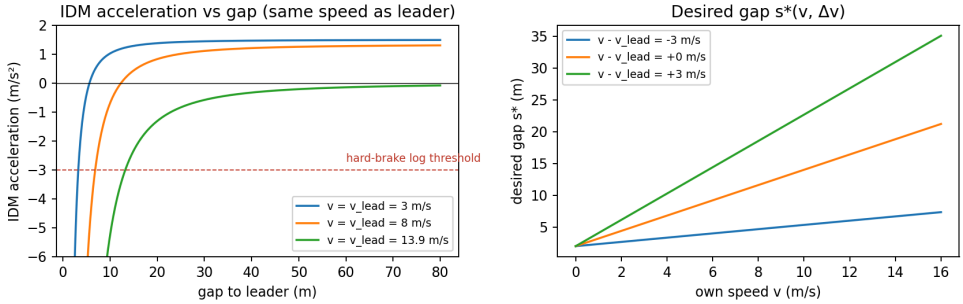


Figure A4. Intelligent Driver Model: acceleration against gap, and the desired gap, at the mean parameters.

3.3 Ballistic update with an exact stop

Positions are advanced with constant acceleration over the step, and a vehicle that would come to rest within the step stops exactly where it does and never rolls back:

$$\text{if } v + a\,\Delta t \geq 0\text{:}\quad s' = s + v\,\Delta t + \tfrac{1}{2} a\,\Delta t^2,\quad v' = v + a\,\Delta t;$$

$$\text{else:}\quad s' = s + \frac{v^2}{2|a|},\quad v' = 0.$$

After all vehicles have moved, an *overlap guard* runs front-first over every lane and connector: if $s_{follower} > s_{leader} - l_{veh} - 0.3$ the follower is pushed back to that position and its speed capped at the leader's; the event is counted (`n_overlap_fixed`). It is a health metric, not a feature: the rate per vehicle-step must stay below 10^{-4} (measured value in Section 10).

3.4 Desired speed, speeding and turn anticipation

On an edge, through traffic uses $v_0^{loc} = v_0$: speeding is allowed, which is what α is for. A vehicle whose next movement is a turn starts slowing up to 120 m before the box so that it reaches the corner at v_{turn}, using half of its own comfortable braking:

$$v_0^{loc} = \min\left(v_0,\ \sqrt{v_{turn}^2 + 2\,(0.5\,b)\,d_{box}}\right), \qquad d_{box} \leq 120\text{ m}.$$

Reading: from 120 m out the driver's target speed falls along a gentle braking curve that ends at the turning speed exactly at the box. The gentle factor and the long zone are there because the IDM lags behind a falling desired speed; they keep the cap continuous for the fastest driver and avoid hard-brake artefacts (Appendix B). Inside a connector, turns are capped at $v_{turn} = 5$ m/s and through movements at $\min(v_0, v_{limit})$.

3.5 Two lanes per direction without lane changing

Figure A7. Lane use for a northbound approach (left-hand traffic)

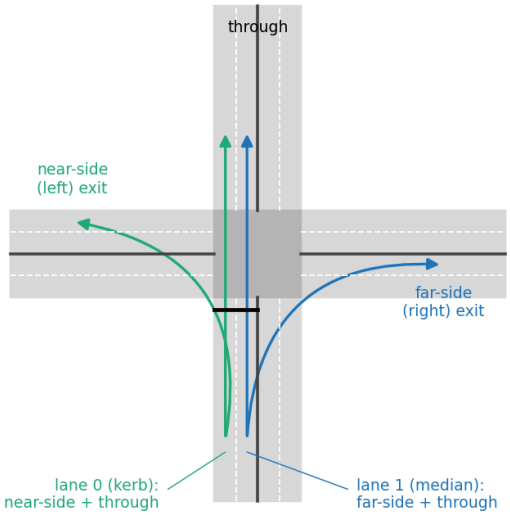


Figure A7. Lane use for a northbound approach under left-hand traffic.

A vehicle's lane is fixed when it enters an edge; there are no mid-link lane changes. At the stop line the kerb lane serves near-side turns and through traffic, the median lane serves far-side turns and through traffic. Lane choice at edge entry follows the vehicle's *next* movement: NS → lane 0, FS → lane $n_{lanes} - 1$, through or boundary-bound → the lane with fewer vehicles (tie → lane 0). The connector delivers the vehicle to whichever lane of the exit edge its *following* movement needs (a "merge inside the box" simplification, stated in the data card); the target lane is planned on the approach, within the 100 m leader horizon of the stop line, counting vehicles already on connectors heading for the same lane, and is not recomputed at box entry. Two vehicles abreast that aim for the same exit lane zipper at the box entry: the one further along leads (ties by vehicle id); while the other is abreast of it, it sees the box edge as a standing leader, so the IDM brings it to rest before the edge end, braking at least at its comfortable deceleration b and harder when the remaining gap demands it (a step below −3 m/s² is logged as a hard brake, as in Section 3.2, and the applied deceleration is clipped at the physical limit b_{emerg}); a residual hold at the edge is counted in `n_merge_holds`, and one whose instantaneous stop would imply braking above the physical limit (speed above $b_{emerg}\,\Delta t = 3.5$ m/s) is also counted in `merge_hard_stops`. A hold between 1.5 and 3.5 m/s implies an abrupt stop of 3 to 7 m/s² that is counted only in `n_merge_holds`; the release has one merge hold in total and no hard stop, so the rule is documented rather than changed. A through vehicle never plans the exit lane already claimed by the head of the other lane ahead of it, re-plans only if the other lane is emptier by more than one vehicle, and keeps its plan within 15 m of the box, so the zipper order cannot flip under it.

3.6 Who is my leader?

The gap g and leader speed v_{lead} in the IDM come from the nearest of these candidates within a 100 m horizon:

- the vehicle ahead in the same lane: $g = s_{ahead} - l_{veh} - s$;
- the stop line, treated as a stationary leader of zero length at s_{stop}, whenever the vehicle "sees" it: it is not committed and either its approach is not green, it has been latched STOPPING, or the lane it needs on the far side is blocked, meaning a queued vehicle ($v < 2$ m/s) leaves less than $l_{veh} + s_0 + 1 = 7.5$ m free at its start, and the vehicle can still stop at b_{stop} (the don't-block-the-box rule, which is anticipatory and colour-independent);
- when it will cross: any vehicle on a connector of the same approach heading for the same exit lane, $g = (L_e - s) + s_{other} - l_{veh}$, or the last vehicle on the target exit lane, $g = (L_e - s) + w_{box} + s_{last} - l_{veh}$;

- inside a connector: a vehicle ahead on the same or a sibling connector to the same exit lane, or the last vehicle on the target exit lane.

Reading: a car will not enter the box if the lane it needs on the far side has no room. Within 100 m of the line, a car that is not committed treats the stop line as a parked car whenever its approach is red, and also on green whenever its exit lane is full; that second rule is the one that does not depend on the light colour. A car that can no longer stop enters anyway and, if it halts inside the box, is counted in `n_box_blocked`.

3.7 Stop or commit at red onset

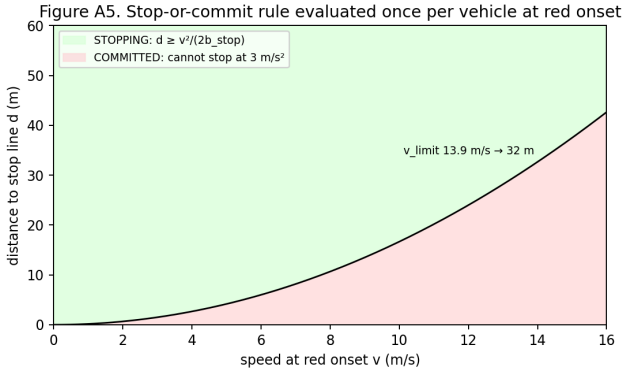


Figure A5. The stop-or-commit rule, evaluated once per vehicle at red onset.

Amber does not exist in this design, so the dilemma it normally resolves is handled explicitly. At the instant an approach loses its green, every vehicle on that approach edge is evaluated once, nearest the stop line first, with its own distance d and speed v:

$$d \geq \frac{v^2}{2\,b_{stop}} \;\Rightarrow\; \text{STOPPING}, \qquad \text{otherwise COMMITTED}, \qquad b_{stop} = 3\ \text{m/s}^2.$$

A STOPPING vehicle takes the stop line as its leader; a COMMITTED vehicle ignores the line, finishes crossing during the all-red, and is counted in `n_red_commits`. Chains of committed vehicles are allowed; a vehicle entering the edge after red onset defaults to STOPPING; green onset clears all latches. A COMMITTED vehicle that nevertheless comes to rest before the line is re-latched STOPPING (counted in `n_commit_revoked`), so a stalled commit cannot freeze the junction, and the COMMITTED exemption from the red rule holds only during the all-red that follows its own red onset. *Reading:* when the light turns red, a car that can stop comfortably before the line does so; one that cannot keeps going and clears during the safety gap.

Worked numbers. The commit distance $v^2/(2b_{stop})$ is 32.2 m at 13.9 m/s, 42.7 m for the fastest car (16.0 m/s) and 50.5 m for an ambulance at its fixed 17.4 m/s (Section 7), which makes ambulances the vehicles most likely to commit at red onset. A vehicle that commits at exactly that distance and keeps its speed needs, to clear the stop-line setback, the box and its own length ($2 + 14 + 4.5 = 20.5$ m beyond the line),

$$t_{clear}(v) \approx \frac{v}{2\,b_{stop}} + \frac{20.5}{v},$$

which is below 4 s only for $v \in [7.4, 16.6]$ m/s and reaches its minimum of 3.7 s at 11.1 m/s. Slow committed vehicles therefore need more than the 4 s minimum all-red, which is why the all-red is **extended until the box is empty** (Section 5) and why the all-red extension histogram has a right tail. Four seconds is a minimum, not a derived safety bound.

3.8 Per-vehicle bookkeeping

`total_wait` accumulates Δt while $v < 2$ m/s; `n_stops` counts transitions from $v > 2$ to $v < 0.5$; `accel_energy` $= \sum a^2 \Delta t$; `n_hard_brakes` and `n_red_commits` as above. These are logged per vehicle

and summed per episode.

4. Demand and routing

4.1 Arrivals: a Poisson process by coin flips

At every step, while spawning is on ($t < 3900$ s), each boundary node e flips a coin:

$$\Pr(\text{new request at } e \text{ in this step}) = p_e = \lambda_e(t)\,\Delta t, \qquad \lambda_e(t) = \lambda_{base}\,\kappa_e\,\pi(t).$$

Reading: every half-second each entrance produces a new car with probability rate × 0.5 s; over many flips this is Poisson traffic (the Bernoulli approximation is accurate while $p_e \le 0.1$, i.e. up to about 720 veh/h per entry). That limit is exceeded only on the north–south entries of `ns_heavy` episodes at stress demand (p_e up to 0.114 after the 1.4 factor and the 1.1 jitter), where arrivals are slightly more regular than Poisson; the effect is confined to the hardest scenario. κ_e is the asymmetry factor: 1 everywhere for the `sym` scenario, 1.4 on the north–south entries and 0.7 on the east–west entries for `ns_heavy`. The released dataset uses a flat profile, $\pi(t) \equiv 1$. A daily profile of hourly multipliers (about 0.1 at night, up to 1.35 in the morning peak and 1.4 in the evening peak) exists for the twin extension of Section 13; flat runs keep their configuration hash unchanged. Per episode, λ_{base} is jittered by a factor $U(0.9, 1.1)$ around its calibrated level.

A request draws its destination and parameters immediately. If the first 10 m of its chosen lane are free and the gap to the last vehicle satisfies $g \ge s_0 + v_{spawn}\,T$, it is inserted at $s = 0$ with $v_{spawn} = \min(v_0, v_{last})$ when a vehicle is within 50 m, else $0.8\,v_0$; otherwise it waits in the node's first-in-first-out entry queue and accrues `entry_queue_delay`.

4.2 Destinations and routes

The destination is drawn from the origin's row of an origin–destination (OD) matrix over the seven other boundary nodes. The default row is relational: the far end of the vehicle's own road 0.30; each end of the parallel road 0.15; each of the four ends of the two crossing roads 0.10 (sums to 1.00). The `uniform` pattern uses 1/7 each. The route is the shortest path on the directed edge graph with turns charged as extra metres,

$$\text{cost(route)} = \sum_{e \in \text{route}} L_e + \sum_{\text{turns}} \text{penalty}, \qquad \text{penalty}_T = 0,\ \text{penalty}_{NS} = 10,\ \text{penalty}_{FS} = 25\ \text{m},$$

with exact ties broken by a coin from the demand RNG stream.

4.3 Free-flow time and delay

$$T_{ff} = \sum_{e} \frac{L_e}{v_0} + n_{turn}\,\frac{w_{box}}{v_{turn}} + n_{thru}\,\frac{w_{box}}{\min(v_0, v_{limit})},$$
$$D = \underbrace{(t_{enter} - t_{request})}_{\text{entry-queue delay}} + \underbrace{(t_{exit} - t_{enter})}_{\text{travel time}} - T_{ff}.$$

Reading: delay is how much longer the trip took than it would have on empty roads with every light green, plus any wait to get onto the road. Unfinished vehicles at the end of the episode count the delay accrued so far. The free-flow time ignores the turn slow-down of Section 3.4 and the re-acceleration after the box, so every turn contributes roughly five seconds of "delay" that is geometry rather than congestion (a lone car on an empty road loses about three seconds slowing on the approach and two regaining speed; listed as a limitation in Section 13).

Gridlock. The rule is evaluated for the network as a whole (no stop-line crossing anywhere for 120 s while at least 10 vehicles are in the network) and for each junction separately (no crossing at that junction for 120 s while at least 10 vehicles wait on its four approach edges). At the first detection the episode ends with `gridlock = True` and is kept as a labelled failure; `gridlock_junction` records which junction fired the rule (−1 for the network-wide rule). This is how the proposal's "gridlock flagged" requirement is met. In the 120-episode run, 3 of 120 episodes end in gridlock (random_dwell at high demand); none under stress.

Spillback locks. Short of gridlock, an approach lane can stand for a long time because its exit lane is blocked by spillback from the next junction. Two liveness measurements are logged per episode without changing any rule: `max_head_wait_s`, the longest time any lane-head vehicle stood still, and `n_spillback_locks` (with `spillback_lock_t` and `spillback_lock_junction`), the number of times an approach lane went 120 s without a stop-line crossing although its approach had been green for at least 40 s within that window; a lock is latched per lane until a crossing or until the lane empties, and is recorded, not acted on. Over the 120-episode run, the longest red faced by an occupied approach is 296 s; the longest time a lane head stood still is 582 s; 13 spillback locks were recorded in 3 episodes.

5. The signal plant: the safety shield

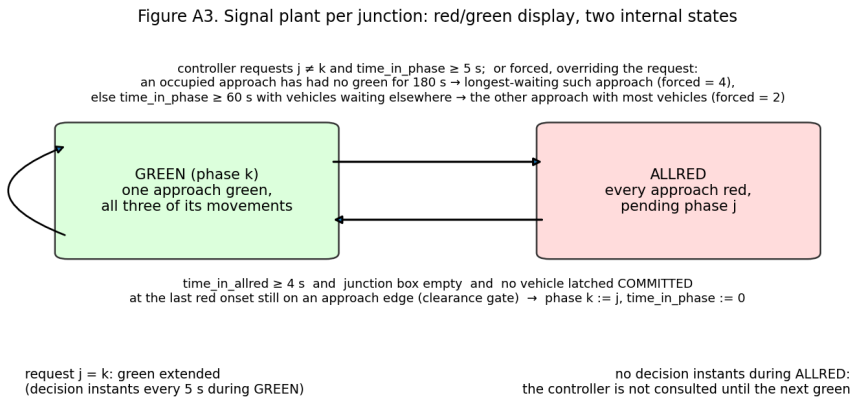


Figure A3. Signal plant per junction: a red/green display driven by two internal states.

Each junction has one *plant*: the safety layer that sits between any controller and the lights, which the proposal calls the safety shield. The display only ever shows red or green; the safety logic lives in two internal states. Every controller, including the learned one, only ever *requests* a phase, and the plant decides when the request can be applied safely, exactly as a real signal cabinet does.

- **State.** `phase` $\in \{0{:}N, 1{:}E, 2{:}S, 3{:}W\}$ (the approach currently or most recently green), `state` $\in \{\text{GREEN}, \text{ALLRED}\}$, `time_in_phase`, `pending_phase`, `n_switches`.
- **Split phasing.** Phase k gives green to all three movements of approach k and red to everything else, so no two green movements can ever conflict and every destination stays reachable.
- **Timings.** Minimum green 5 s, maximum green 60 s, all-red at least 4 s, a starvation guard of 180 s (three maximum greens), one decision instant every 5 s during green (the first at green onset + 5 s), none during all-red.

Switch procedure at a decision instant with request j.

1. **Starvation guard.** If an approach with vehicles has had no green for 180 s or more (since its green last ended, or since the start of the episode), the plant forces a switch to the longest-waiting such approach (ties in rotation order) and records `forced = 4`. This is the guarantee behind the proposal's fairness requirement: whatever the controller asks for, an occupied approach gets its green within the guard plus one decision interval and one all-red. It pre-empts the max-green rule below, which on its own can starve a third approach because it always picks the largest count. The guard bounds the time to a *green*; the time a lane head waits behind a blocked exit lane is a separate matter (Section 13).
2. **Max green.** Otherwise, if `time_in_phase` $\geq$ 60 s and some other approach has vehicles, the plant forces a switch to the other approach with the most vehicles (ties in rotation order) and records `forced = 2`. Max green is not a metronome: with nobody waiting elsewhere, green is simply held.
3. If j equals the current phase, green is extended; the next decision is in 5 s.

4. Otherwise the plant enters ALLRED with `pending_phase` $= j$, runs the stop-or-commit evaluation on the approach that lost green, and logs `switched = True`.
5. Green for j begins at the first step t with $t - t_{allred} \geq 4$ s **and** no vehicle on any of the junction's twelve connectors **and** no vehicle that was latched COMMITTED at the previous red onset still on an approach edge of the junction (the clearance gate; committed vehicles enter regardless of colour). The extension beyond 4 s is logged (`allred_extension`).

There are no decision instants during ALLRED, so the controller is not consulted again until the next green. Because decision instants exist only during green and start at +5 s, the forced codes for "min green" (1) and "all-red" (3) are structurally unreachable; only codes 0 (none), 2 (max green) and 4 (starvation guard) occur in the data. The episode row records the longest red an occupied approach faced (`max_approach_red_s`) and the number of guard switches (`n_forced_starvation`).

Safety invariants, checked by the test suite and counted in every episode (`n_invariant_violations` must be 0): never two approaches green; box empty, and no committed vehicle still upstream, at every green onset; no stop-line crossing on red unless COMMITTED during the all-red that follows its own red onset (a crossing while another approach is green is a violation whatever the latch); minimum green respected; all-red ≥ 4 s; vehicle conservation (requests = arrived + in network + still queued at entry). Every controller's request passes through this plant, so every controller is safe by construction and the comparison between them is fair. This is what the proposal means by safety-shielded requests, and it is why a learned rule can sit behind an existing signal controller's own interlocks without a separate safety layer.

Effective green, lost time, capacity. Per phase the lost time is start-up lost time plus all-red, $l = l_{startup} + r_{all} \approx 2 + 4 = 6$ s, so

$$g_{eff} = g + r_{all} - l = 20 + 4 - 6 = 18 \text{ s}, \qquad C = 4\,(g + r_{all}) = 96 \text{ s},$$
$$c = s_{sat}\, n_{lanes}\, \frac{g_{eff}}{C}, \qquad x = \frac{q}{c}.$$

Reading: of each 24 s slot about 18 s actually move cars; two seconds are lost getting rolling and four are the safety gap. With the measured saturation flow of 1,631 veh/h/lane the formula gives $c_{formula} = 612$ veh/h per two-lane approach; the simulated junction realises $c = 549$ veh/h of it under fixed time (Section 9), which is the capacity the demand levels are calibrated against. The 18 s effective green is a fixed design constant used for capacity, calibration and Webster's term; it is not recomputed from the measured start-up lost time of 1.29 s (which would give 18.7 s). The measured lost time is used where it matters for control: in the teacher's clearing time of Section 7. Webster's (1958) uniform-delay term, used only as a sanity check at low load, is

$$d_1 = \frac{C\,(1 - g_{eff}/C)^2}{2\,(1 - x\, g_{eff}/C)}.$$

6. Controllers and label functions

Every controller maps the logged feature row to a requested approach. The plant (the safety shield) enforces safety, so a controller is just a rule.

- **fixed_time.** Rotate N→E→S→W, requesting the next approach once `time_in_phase` ≥ 20 s; it serves empty approaches, which is what fixed time does. This is the deployed default that Webster-style timing plans produce.
- **max_pressure** (the teacher on rows without an ambulance; Section 7 adds the ambulance rule that completes the label rule). For approach a let X_a be its three exits minus those whose target edge is blocked (Section 3.6). Then

$$p_a = n_a - \frac{1}{|X_a|} \sum_{d \in X_a} n_d^{down}, \qquad p_a = -10^9 \text{ if } X_a = \varnothing,$$

$$\text{request} = \begin{cases} \operatorname{argmax}_k p_k & \text{if } p_{best} > p_{phase} + 2 \\ \text{phase} & \text{otherwise,} \end{cases}$$

with ties to the lowest index and the max-green rule applied first. *Reading:* an approach's pressure is how many cars are on it minus how full the roads it would send them onto are; an approach whose every exit is full cannot be served; a two-vehicle hysteresis stops the lights flickering. This is a max-pressure-style heuristic in the spirit of Varaiya (2013), not the per-movement form, and it is throughput-optimal only under idealised assumptions this network violates. It is the proposal's main non-priority baseline, and the paired comparison of Section 10 (Experiment 5) shows where it wins and loses: with the specified hysteresis and minimum green it switches about four times a minute, beats fixed time at design demand and loses to it at high demand, because every switch costs about 5 s of lost time (the 4 s minimum all-red plus the measured 1.29 s start-up lost time of Section 5). The label `y_mp_action` is therefore a strong parameter-light heuristic, not an optimum; the hysteresis and the minimum green are configuration values if a calmer teacher is wanted.

- **random_dwell.** Hold a random $U(5, 40)$ s, then request a random other approach. Used only to put rare states (starved approaches, spillback) into the dataset; train split only.
- **max_pressure_eps.** With probability $\varepsilon = 0.3$ request a uniformly random approach, else max-pressure.
- **preempt_naive** and **priority_mp.** The ambulance rules of Section 7. Neither drives a dataset episode: the behaviour mixture of Section 8.4 has four controllers, and these two are run only in the evaluation stage. The teacher therefore enters the dataset only as the label `y_mp_action`, evaluated on the states that the four behaviour controllers visited.
- **The learned controller.** The gradient-boosted model of Section 12.3, called through the same interface in the modelling stage; it is not a behaviour controller of the dataset, it is what the dataset is for.

A longest-queue rule (switch when another approach holds more than three vehicles more than the current one) is also implemented; it is the controller under which the queue-discharge experiment of Section 10 runs, where it holds the green on the only occupied approach. It drives no dataset episode and is not evaluated.

Labels. Throughout, *teacher* and *label rule* mean the same thing: the rule `priority_mp` (Section 7), whose request is logged as `y_mp_action` on every row from the row's own columns, and which the supervised model of Section 12.3, the proposal's supervised signal controller, is trained to imitate. On rows without an ambulance it is exactly the max-pressure request. It is the *request*, not the applied phase, so it can differ from `action_applied` on max-green rows even under the max-pressure controller. `y_q_total_t30` is the junction's queued-vehicle count 30 s later, read from `junction_1hz` at the first whole second $\geq t + 30$; it is NaN on the last 30 s of the main period, during cool-down, and after a gridlock stop. No other label is logged; in particular there is no per-row ambulance-delay-ahead target and no counterfactual per-phase outcome (Section 12.3 explains how the proposal's model goal is met without them).

Why a mixture of behaviour controllers. Under one deterministic policy, states it avoids never occur, and the applied action is a deterministic function of the state, so a model of "queue 30 s ahead given state and action" cannot separate the effect of the action from the state. Random-dwell and ε-greedy episodes supply both actions at similar states; fixed time supplies long queues. The imitation label is defined on every row regardless of which controller drove, so the classifier sees the rule evaluated on off-policy states. Regression targets are "under the logged policy" and are described as such. One gap is stated openly: none of the four behaviour controllers grants ambulance priority, so the rows on which `y_mp_action` asks for the ambulance's approach are states produced by non-priority control, and the states that follow an early switch for an ambulance occur only when ε-greedy or random-dwell happens to produce one. The closed-loop evaluation of Section 12.3 is therefore also a test of how the learned

controller behaves in states it rarely saw; adding teacher-driven episodes to the mixture is a candidate change for the next generator version.

7. The ambulance extension (healthcare scenario)

The business setting is the proposal's: Jakarta's Area Traffic Control System (ATCS), operated by UP.SPLL of the Transport Agency (Dishub DKI), can give an ambulance of the Health Agency's 119 fleet a green, but today a control-room operator decides each case by hand (UP.SPLL Dinas Perhubungan Provinsi DKI Jakarta, n.d.). The healthcare value of the project is ambulance response time, where every minute counts (Larsen et al., 1993) and Jakarta's median response is long (Brice et al., 2022); the intervention is emergency-vehicle priority at signals, decided by a rule learned from this dataset instead of by an operator. In the deployment the proposal describes, ambulance GPS is the only new input a city has to add to its existing signal control, so the simulator models the ambulance channel as a GPS-style feed and nothing else. The extension keeps every car-side rule unchanged so that datasets with and without ambulances are comparable.

Naming. Column, stream and scenario names use the prefix `ev` for *emergency vehicle*, the general term of the preemption literature and of the proposal's title. Every emergency vehicle in this dataset is an ambulance (`vehicle_type = "ambulance"`), and the text says ambulance throughout.

Spawning. A fifth random stream (`ev`) drives everything about ambulances, so cars are bit-identical with ambulances on or off for the same seed. The scenario factor `ev_rate` is "0" (30% of episodes) or "default" (70%). At "default" every boundary node flips a second coin each step,

$$p_{ev} = \frac{\lambda_{ev}}{3600}\,\Delta t, \qquad \lambda_{ev} = 4 \text{ ambulances/h per node},$$

independent of car demand. That rate is far above reality and is chosen for statistical power (about 34 ambulances per episode in the measured run); it is stated in the data card. Destination and route come from the same OD row and the same shortest-path rule.

Ambulance parameters (fixed, no α draw): $v_0 = 1.25\,v_{limit} = 17.4$ m/s, $a_{\max} = 2.5$, $b = 3.0$ m/s², $T = 0.9$ s, $l_{veh} = 6.0$ m. Same IDM, same stop-or-commit rule, same lanes and box rules. **Ambulances never run a red light**: their advantage can only come from the signal, which keeps every safety invariant intact and makes the signal controller the object of study. Without lane changing an ambulance behind a queue waits for the queue to discharge. The proposal treats this as Jakarta's geometry rather than a defect: with two full lanes and no shoulder there is no room to yield, so the signal is the only lever. What remains a stated limitation is that cars cannot pull aside even where a real street would let them.

What a controller can see (GPS-style channel, no sensor noise), per approach a: `ev_present_a`; `ev_dist_a`, the distance of the nearest ambulance to the stop line (−1 if none; the 300 m horizon never binds here because every approach edge is shorter than 300 m, so `ev_present_a` means an ambulance is somewhere on the approach edge); `ev_eta_a` $= d/\max(v_{ev}, 3\text{ m/s})$; and `ev_q_ahead_a`, the number of queued vehicles ahead of it in its lane. These four columns are the only input the design adds to what an existing signal controller already measures (the queue and occupancy counts of Section 8.2): the proposal's ambulance GPS as the only new input. The 119 fleet's vehicle tracking would supply them in a deployment; in the twin they are read without noise or delay (Section 13).

Naive full preemption (`preempt_naive`): request the approach of the ambulance with the smallest ETA whenever that ETA is at most 20 s; otherwise behave as max-pressure. This is the "force green for the ambulance" strategy of the preemption literature (Nelson & Bullock, 2000; Chen & Wang, 2024; Roy et al., 2024; Su et al., 2023), the immediate forced green of current practice, and the proposal's third baseline.

Priority-aware teacher (`priority_mp`, the label rule): request the ambulance's approach only when it will arrive within the time the queue ahead of it needs to discharge,

$$\mathrm{ETA}_a \le t_{clear,\,a} = r_{all} + l_{startup} + q_a^{ahead}\, h_{sat},$$

with the measured start-up lost time and saturation headway; before that it keeps serving max-pressure, and after the ambulance has crossed it returns to max-pressure. *Reading:* switch exactly early enough that the last car in front of the ambulance has rolled across the line when the ambulance reaches it, and not a second earlier. This is the proposal's ETA-aware pre-clearing, written as a formula so that it can be inspected and imitated; it is what the crews' need for fewer stops and a predictable passage turns into at the junction.

New measurements. `vehicles.csv` carries `vehicle_type` (`car` or `ambulance`). `episodes.csv` adds `n_ambulances` (requests in the whole episode) and, over ambulances requested in the main period (NaN if none): `ev_mean_travel_time_s`, `ev_mean_delay_s`, `ev_p90_delay_s` and `ev_mean_stops`; `car_mean_delay_penalty_s`, the proposal's cost to general traffic, defined as the mean car delay minus the same quantity in a paired run of the same seed and controller with no ambulances, which the generator runs automatically (the non-priority cost that Chen and Wang (2024) optimise against); and `recovery_time_s`, which, unlike the four ambulance KPIs, is not restricted to the main period: it is averaged over every (ambulance, junction) pair of the whole episode, and its recovery window runs to the end of the episode. For the recovery time, each (ambulance, junction) pair records the junction's `q_total` the moment the ambulance enters one of its approach edges, q_0; after the ambulance's front bumper leaves the box at t_{clear}, the recovery time is the first whole second $t \ge t_{clear}$ at which $q_{total}(t) \le 1.1\, q_0$, minus t_{clear}. The episode value is the mean over pairs; pairs whose queue has not returned to that level by the end of the episode are left out, so the metric is optimistic when recovery is slow. *Reading:* how long after the ambulance has gone until the junction looks the way it did when the ambulance appeared; this is the "return to coordination" that Nelson and Bullock (2000) found decides whether preemption disrupts a corridor.

Evaluation protocol (proposal Goal 4). The evaluation is a separate modelling-and-evaluation stage, not part of dataset generation: `preempt_naive` and `priority_mp` are implemented but are not in the behaviour mixture of Section 8.4, so no episode of the released dataset is driven by them, and the ambulance KPIs in the data card are grouped over the four behaviour controllers only. The evaluation grid is five controllers {fixed_time; max_pressure, the proposal's main non-priority baseline and the reference for the bound on car delay (Section 10, Experiment 5, shows it loses to fixed time at high demand); preempt_naive, the proposal's naive full preemption; priority_mp, the teacher and the reference for the imitation; the learned controller} × two demand levels {design, high} × at least eight seeds, with ambulances on and the same seed, hence the same cars and the same ambulance requests, under every controller. The **primary metric** is ambulance travel time through the grid, `ev_mean_travel_time_s`, as the proposal states; `ev_mean_delay_s` is reported beside it. Because paired runs give every ambulance the same route, and hence the same free-flow time, under every controller, the paired difference in `ev_mean_delay_s` between two controllers equals the paired difference in travel time plus the difference in entry-queue wait, which is close to zero at design demand but large once a controller lets queues back up to the entry nodes, so at high demand the two metrics can rank controllers differently and are never used interchangeably. The pilot tables of Section 12.5 lead with `ev_mean_delay_s`; the per-episode results also hold `ev_mean_travel_time_s`, and the report's evaluation table carries both, travel time first. **Secondary metrics** are the general delay of cars (`mean_total_delay_s`, compared with max-pressure on the same seed; its paired form against the no-ambulance run is `car_mean_delay_penalty_s`), `recovery_time_s`, and the fairness spread defined in Section 12.3. The proposal's **stated bound on general traffic** is operationalised there as well: the learned controller must keep car delay within a margin of max-pressure, and the margin proposed as the group's operating point is 10%. Differences are paired by seed and reported as 95% t-intervals (Section 12.3). Results are reported in Section 12.5. The pilot results of Section 12.5 were computed on the dataset this document describes (same generator version and configuration hash).

Who reads which column. The proposal's stakeholder table maps onto the dataset as follows.

Stakeholder (need)	Where the dataset answers it
Ambulance crews and patients (fewer stops, predictable passage)	`ev_mean_travel_time_s`, `ev_mean_delay_s`, `ev_p90_delay_s`, `ev_mean_stops`; the ETA-aware teacher above
Traffic-control operators (no manual intervention, no gridlock)	every request passes the plant of Section 5; `gridlock`, `gridlock_t`; `down_blocked_*` in `signal_ticks`
General road users (fair, bounded delay)	`mean_total_delay_s` against the bound, `car_mean_delay_penalty_s`, `recovery_time_s`; delay by entry node from `origin` and `total_delay_s` in `vehicles.csv`
City government (evidence before investment)	paired seeds with common random numbers (Section 8.3) and t-intervals (Section 12.3)
Researchers and students (reusable data)	the CSV tables, the data dictionary, the data card and the download link (Sections 8 and 12.4)

8. Dataset schema

8.1 Tables and grain

File	One row per	Primary key	Rows in the 120-episode run
`episodes.csv`	episode	`episode_id`	120
`vehicles.csv`	vehicle (car or ambulance)	`episode_id, veh_id`	360,443
`signal_ticks.csv`	junction × decision instant (**the ML table**)	`episode_id, junction_id, t`	300,598
`junction_1hz.csv`	junction × second	`episode_id, junction_id, t`	1,996,932
`network_edges.csv`	directed edge	`edge_id`	24
`vehicle_states_<id>.csv`	vehicle × second (showcase episodes only)	`episode_id, t, veh_id`	optional

Plus `data_dictionary.csv` (every column of every table with type, unit, description and namespace), `config.json` (the full configuration including the calibrated demand levels) and `DATACARD.md`, a datasheet in the sense of Gebru et al. (2021). These files, with their SHA-256 checksums (Section 12.4), are what the download link of the report points to.

8.2 The ML table, `signal_ticks` (80 columns)

One row per junction per decision instant. Columns are grouped by namespace; the namespace is recorded in the data dictionary so that feature sets can be selected by rule rather than by hand. The full column-by-column dictionary is Appendix C.

Namespace	Columns	What they hold
key, time	`episode_id, junction_id, t, tick_idx, is_warmup, is_cooldown, is_tail`	identity of the row and the period flags
signal	`phase, time_in_phase_s, max_green_remaining_s`	the plant's own state
state (per approach N, E, S, W)	`q_a, n_a, q_a_l0, q_a_l1, arr_a_30s`	queued and total vehicles on the approach edge, per lane, and arrivals in the trailing 30 s
downstream (per exit direction)	`down_n_d, down_occ_d, down_blocked_d`	vehicles on the outgoing edge, its occupancy relative to storage, whether its start is blocked
neighbour (per direction)	`nb_d_exists, nb_d_phase, nb_d_green_toward_me`	whether a neighbouring junction exists there, its phase, and whether its green feeds this junction
sensor	`obs_q_a` (×4)	the noisy queue count defined below
ev (per approach)	`ev_present_a, ev_dist_a, ev_eta_a, ev_q_ahead_a`	the ambulance channel of Section 7
action	`action_requested, action_applied, forced, switched`	what the controller asked, what the plant did
label	`y_mp_action, y_q_total_t30`	the imitation target and the 30 s-ahead queue

Flags. `is_warmup` ($t < 300$), `is_cooldown` ($t \geq 3900$), `is_tail` (last 30 s of the main period, where the 30 s-ahead label would look into cool-down). Training and evaluation use rows with none of the three set.

Sensor copy. For each approach the observed queue is a noisy version of the true one,

$$\tilde{q}_a = \text{Binomial}(q_a,\, 1 - p_{miss}) + \text{Poisson}(p_{miss}/2), \qquad p_{miss} \in \{0, 0.1\},$$

drawn from the sensor stream; with $p_{miss} = 0$ it equals q_a. Models trained on `obs_q_*` instead of `q_*` quantify how much control quality survives realistic sensing (a camera-based queue count of the kind an area traffic control centre already has), and the twin's estimator later writes into the same columns.

Leakage rules. No feature depends on anything after t; every window is trailing. The pressure values that generate the label are not stored as columns. There is no clock feature (no time of day, no cumulative counters) so a model cannot memorise the demand schedule. `junction_id` is a key, not a feature: the corner a junction sits in is already implied by which neighbours exist.

8.3 Seeds, streams and determinism

$$\text{seed} = 100000 + \text{episode_id}, \qquad \text{seed} \rightarrow \text{five independent child streams } (\text{demand}, \text{vehicles}, \text{controller}, \text{sens}$$

The five streams are spawned from the episode seed by a hash-based seed-sequence scheme, so they are statistically independent, and changing how one stream is consumed (switching ambulances off, or running a different controller) leaves the draws of the other four unchanged. Arrivals, destinations and route ties come from the demand stream; driver parameters from the vehicles stream; ε draws and random dwells from the controller stream; `obs_q` noise from the sensor stream; everything about ambulances from the ev stream. Two controllers run on the same seed therefore face identical cars (common random numbers), which is what makes the proposal's paired comparisons meaningful. The determinism test runs an episode twice and asserts identical `vehicles.csv`; `config_hash` (first 12 hex characters of the SHA-1 of the configuration JSON and generator version) and `generator_version` are stamped on every episode row.

8.4 Scenario mixture, splits and preprocessing

Each episode draws, stratified so that the realised shares match the targets: demand level {low 25%, design 40%, high 25%, stress 10%}; asymmetry {sym, ns_heavy}; OD pattern {default 70%, uniform 30%}; controller {max_pressure 40%, max_pressure_eps 30%, fixed_time 20%, random_dwell 10%}; sensor noise {0: 60%, 0.1: 40%}; ambulance rate {0: 30%, default: 70%}. Splits are assigned per episode, never per row: 70/15/15 train/validation/test stratified by (demand level, controller); every **stress** episode goes to `test_ood` and is never trained on; random-dwell episodes are train-only. Consecutive rows five seconds apart are near-duplicates, so a row-wise split would report an accuracy that collapses in closed loop.

Preprocessing. The proposal names three preprocessing steps; all three are done by the generator so that the published CSV files are analysis-ready. *Warm-up filtering:* rows and vehicles are flagged (`is_warmup`, `is_cooldown`, `is_tail`, Section 8.2) rather than deleted, and every consumer applies the same filter. *Episode-level splits:* the `split` column above. *Label post-pass:* `y_mp_action` and `y_q_total_t30` are computed after the episode ends (Sections 1 and 6). The analysis in R (Section 12.4) starts from the flag filter and needs no further cleaning. Because every episode is an independent seed, generation and preprocessing parallelise trivially across episodes: a release is one independent job per episode whose only shared input is the configuration.

Episodes of the 120-episode run (every episode is listed for a pilot; a release is summarised by mix, split and mean KPIs):

Episode mix (count by demand level and controller):

demand_level	controller	episodes
design	fixed_time	9
design	max_pressure	24
design	max_pressure_eps	10
design	random_dwell	5
high	fixed_time	9
high	max_pressure	4
high	max_pressure_eps	13

demand_level	controller	episodes
high	random_dwell	4
low	fixed_time	4
low	max_pressure	15
low	max_pressure_eps	8
low	random_dwell	3
stress	fixed_time	2
stress	max_pressure	5
stress	max_pressure_eps	5

Split by demand level (episodes):

split	design	high	low	stress
test	7	4	4	0
test_ood	0	0	0	12
train	35	22	22	0
val	6	4	4	0

Mean KPIs by demand level:

demand_level	mean_total_delay_s	p90_total_delay_s	throughput_vph	switches_per_min	n_red_commits	ev_mean_delay_s	car_mean_delay_penalty_s	recovery_time_s
low	62.9	118.8	1,556.7	3.5	272.0	62.1	0.1	32.2
design	101.5	173.6	2,675.2	3.8	730.0	99.8	-0.0	76.5
high	349.5	597.5	3,082.1	3.7	1,051.5	362.0	4.8	77.5
stress	667.8	1,165.6	3,201.1	4.3	1,325.4	688.1	14.6	59.4

Mean KPIs by controller:

controller	mean_total_delay_s	p90_total_delay_s	throughput_vph	switches_per_min	n_red_commits	ev_mean_delay_s	car_mean_delay_penalty_s	recovery_time_s
fixed_time	177.7	312.3	2,847.8	2.5	337.7	149.3	1.7	88.7
max_pressure	161.2	276.2	2,426.4	4.0	709.4	172.7	3.3	53.4
max_pressure_eps	292.3	500.3	2,620.3	4.8	1,269.0	315.4	7.3	66.7
random_dwell	227.9	420.7	2,236.8	2.0	234.4	196.4	-9.8	71.7

9. Demand calibration from the measured saturation flow

The demand levels are defined by load, not by a textbook number. The chain is: measured saturation headway → saturation flow → formula capacity → measured capacity → target volume-to-capacity ratio → arrival rate per entry.

1. **Saturation headway** from the queue-discharge experiment (Section 10, Experiment 1; the calibration chain is given first because it is part of generation): the mean headway of vehicles 5 to 15 of a 20-vehicle standing queue after green,
$$h_{sat} = \frac{1}{11}\sum_{i=5}^{15}\left(t_i - t_{i-1}\right), \qquad s_{sat} = \frac{3600}{h_{sat}},$$
and the start-up lost time is the sum over the first four headways (vehicles 1 to 4) of their excess over h_{sat}. *Measured:* $h_{sat} = 2.21$ s, $s_{sat} = 1631$ veh/h/lane, $l_{startup} = 1.29$ s.
2. **Formula capacity** per two-lane approach under fixed time: $c_{formula} = s_{sat} \cdot 2 \cdot 18/96 = 612$ veh/h. It counts a through-only single-lane discharge, which the simulated junction cannot sustain on two lanes with zipper merging at the box and turning vehicles capped at v_{turn}.
3. **Measured capacity.** One saturated single-junction probe episode under fixed time (every approach fed at 1.3 times the formula capacity for 1200 s after warm-up) measures what the junction actually passes: $c = $ main-period throughput / 4 approaches $= 549$ veh/h, a realisation of 90% of the formula value. Both numbers and their ratio are saved with the dataset.

4. **Flow per unit of base rate.** From the OD matrix and the routes, the flow on each approach edge is a fixed multiple of λ_{base},
$$F_e = \sum_o \sum_d \lambda_{base}\, \kappa_o\, P(d \mid o)\, \mathbb{1}[e \in \text{route}(o,d)] = \lambda_{base}\, f_e,$$
and the busiest internal approach has $f_{\max} = 1.18$ (edge `J00_J01`, read from-node to-node as in Section 2).
5. **Base rate per level** from the target ratio $x_\ell \in \{0.4, 0.7, 0.95, 1.15\}$ for {low, design, high, stress}, using the measured capacity:
$$\lambda_{base}(\ell) = \frac{x_\ell\, c}{f_{\max}},$$
giving low 186, design 326, high 442, stress 535 veh/h per entry node. The calibration step writes these into the configuration saved with the dataset; every episode row carries `measured_sat_flow_vphpl`. The single-junction sweep of Section 10 runs the same simulator as the probe, so their agreement (low 0.40, design 0.70, high 0.95, stress 1.15) checks the calibration arithmetic, not the capacity itself; the independent evidence for the capacity is the saturation-flow measurement and the fixed-time delay curve of Section 10. The calibration is also single-junction: on the 2×2 grid the nominal levels are not sub-capacity for every controller (Section 10, Experiment 5, and Section 13).

Reading: "design" demand means the busiest approach runs at 70% of what a fixed-time signal actually carries in this simulator, measured rather than assumed. The calibration uses the default OD row with symmetric demand ($\kappa_e = 1$); under `ns_heavy` the north–south stubs carry 1.4 times the base rate, so a "high" north-south-heavy episode loads those stub approaches above the fixed-time capacity of a single approach (nominal $x = 1.4 \times 0.95/f_{\max} \approx 1.13$ before jitter, about 1.36 at stress), which is intended as a hard case and is visible in the KPIs.

10. Validation experiments and results

The five experiments below are run before any dataset is generated; they write the figures reproduced here and a machine-readable validation summary from which the numbers in this document are filled.

Experiment 1: queue discharge. Twenty vehicles stand in one lane on a single-junction grid; the light turns green; stop-line crossing times are recorded.

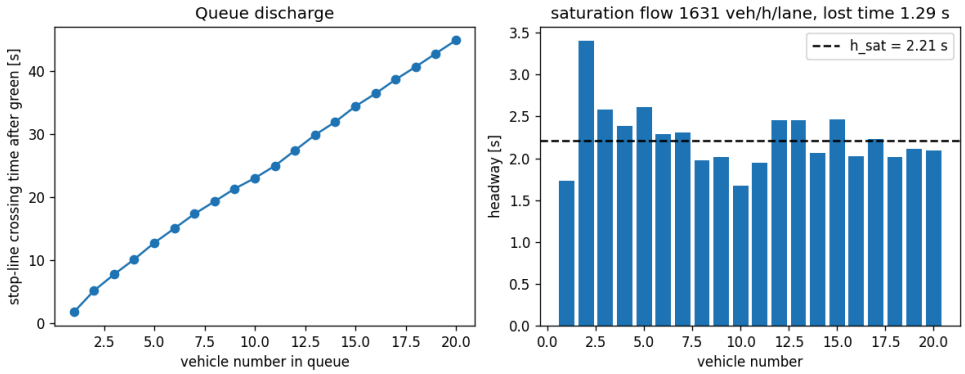


Figure V1. Queue discharge: crossing times and headways.

Result: $h_{sat} = 2.21$ s, saturation flow 1,631 veh/h/lane (acceptance band 1600–1900), start-up lost time 1.29 s. Pass.

Experiment 2: delay versus demand on one junction under fixed time. The arrival rate is swept so that the busiest approach runs at a nominal x from 0.2 to 1.1; mean total delay per vehicle must rise smoothly and blow up near capacity, and at low load should track Webster's (1958) uniform-delay term. The sweep also measures the capacity at which the busiest approach saturates; it agrees with the probe

of Section 9 (ratios of the four levels low 0.40, design 0.70, high 0.95, stress 1.15), which is a consistency check of the calibration rather than an independent validation.

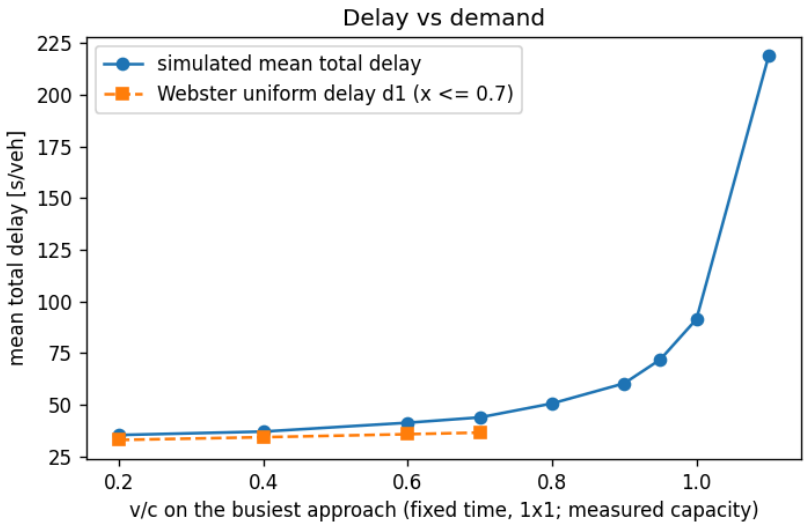


Figure V2. Delay versus volume-to-capacity ratio.

x = v/c	λ per entry (veh/h)	mean delay (s)	p90 delay (s)	throughput (veh/h)	max queue	Webster d1 (s)
0.20	110	35.3	72.8	449	10	32.9
0.40	220	37.0	72.1	902	16	34.3
0.60	329	41.3	73.6	1337	25	35.7
0.70	384	43.8	76.5	1553	33	36.5
0.80	439	50.6	85.1	1769	45	n/a
0.90	494	60.3	100.9	1969	60	n/a
0.95	522	71.9	124.8	2055	67	n/a
1.00	549	91.4	165.8	2130	98	n/a
1.10	604	218.8	396.9	2201	136	n/a

Experiment 3: one design-demand hour on the 2×2 grid with max-pressure. All invariant counters must be zero and delay plausible.

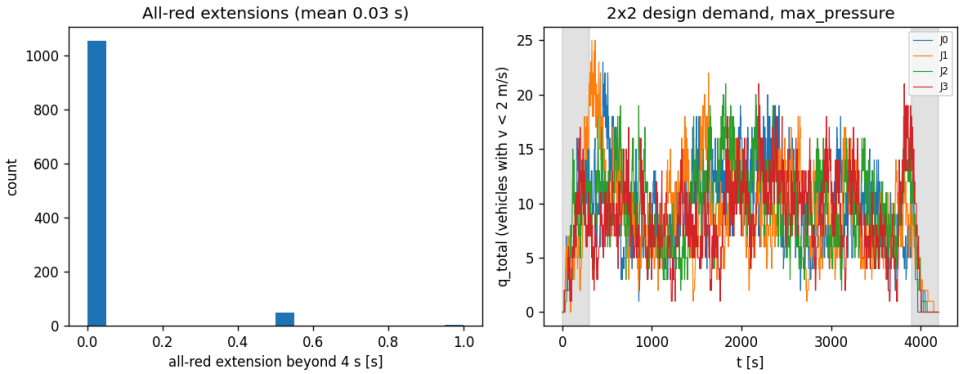


Figure V3. All-red extensions and queue trajectories.

Result: mean total delay 69.5 s per vehicle (plausibility band 30–90 s), 90th percentile 116.9 s, throughput 2,657 veh/h, mean all-red extension 0.03 s, invariant violations 0, runtime 1.8 s for 4200 simulated seconds. Switch rate 4.2 per minute per junction against an acceptance band of 0.7–3.5: the specified teacher flickers above the band, a known property discussed in Section 6. Hard brakes per red onset 0.63, of which a share of 39% were clipped at the physical limit: a STOPPING vehicle near the stop-or-commit boundary brakes harder than its comfortable rate, so this is a property of the specified

rule and a health metric, not a comfort KPI. Longest red faced by an occupied approach 140 s, starvation-guard switches 0, merge holds 0 (hard merge stops 0), commits revoked 0, longest standing lane head 166 s, spillback locks 0. The grey bands in the right panel are warm-up and cool-down; the queues settle within the warm-up.

Experiment 4: ring-road fundamental diagram. Vehicles on a closed 1 km loop at increasing density; flow must rise to a capacity near the saturation flow and fall to zero at the jam density $1000/(l_{veh} + s_0) = 154$ veh/km/lane. The IDM (Treiber et al., 2000) gives a concave free branch and a roughly linear congested branch, so "approximately triangular" is the expected shape.

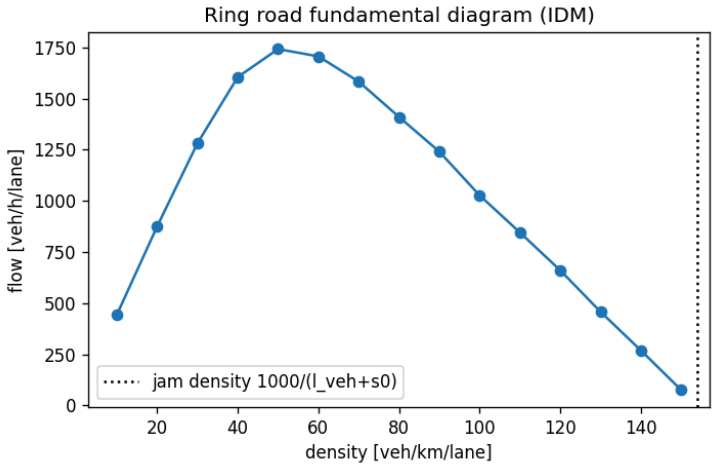


Figure V4. Ring-road fundamental diagram.

Experiment 5: max-pressure against fixed time on paired seeds. The two rule-based controllers of Section 6 are run on the same seeds at design and high demand (three seeds each in the validation run); the paired difference in mean total delay is reported with its 95% *t* half-width.

Demand level	Seeds	Max-pressure mean delay (s)	Fixed-time mean delay (s)	Paired difference, max-pressure minus fixed time (s)	Throughput difference (veh/h)	Max-pressure switches/min
design	3	69.4	100.3	-30.9 ± 0.7	-12	4.2
high	3	317.3	152.7	+164.6 ± 70.5	-345	4.4

Max-pressure wins at design demand and loses at high demand; Section 6 explains why and Section 12.3 draws the consequence for the learned controller. The same two controllers on the full 2×2 grid, one seed per level, show where the network stops clearing its load (vehicles in the network 900 s into the episode and at its end):

Level	Controller	Mean delay (s)	Throughput (veh/h)	In network at 900 s	In network at end	Queues grow
design	fixed_time	99.2	2606	67	76	no
design	max_pressure	73.5	2589	44	62	no
high	fixed_time	149.6	3473	142	141	no
high	max_pressure	305.9	3106	144	434	yes
stress	fixed_time	375.1	3668	227	581	yes
stress	max_pressure	640.8	3143	336	536	yes

Dataset health. Over the 120 episodes of the measured run: invariant violations 0; overlap-guard rate 0.00e+00 per vehicle-step; red commits per episode 123–1803; gridlock episodes 3; label balance of `y_mp_action` over main-period rows: N 27%, E 22%, S 28%, W 22%; share of rows where the label asks for a switch: 61%.

11. Step-by-step: generating the dataset

The generator is a small Python package (Python 3.11 or newer with numpy, pandas, matplotlib and pytest; scikit-learn for the models). Generating a release is five steps. The first three are each a single command: the test suite, the validation experiments and the generator. The last two are done by hand: a check of the output and the publication.

1. **Test.** Run the unit-test suite: geometry (the heading-rotation test of Section 2), IDM, stop-or-commit, plant timing, the safety invariants, determinism, label recomputation, the sensor copy, and the ambulance tests (teacher rules on hand-made rows, ambulance spawning and observation, common random numbers for cars with and without ambulances).
2. **Validate.** Run the five validation experiments of Section 10. They measure the saturation flow and start-up lost time, sweep demand, run the design episode and the ring road, compare max-pressure with fixed time on paired seeds, and save the figures and the validation summary.
3. **Generate.** Run the generator with the number of episodes and an output folder. It first calibrates the four demand levels from the measured saturation flow and a capacity probe episode (Section 9), then runs the episodes of the stratified scenario mixture (Section 8.4), the paired no-ambulance run of every episode with ambulances, and writes the tables of Section 8, the configuration, the data dictionary and the data card. A 1 Hz trajectory log of chosen showcase episodes can be added for animation and for the offline replay of Section 13; such logs are optional extras, not covered by the checksum file and not counted in the file sizes quoted in this document. Further episodes can be generated later as a separate batch with a starting episode id beyond the existing ones, written to their own folder and concatenated by hand; existing seeds do not change, because the seed is a function of the episode id (Section 8.3), but the scenario mixture and the split fractions are stratified within each batch, not across the union.
4. **Check.** Every row of `episodes.csv` must have `n_invariant_violations = 0` and the same `config_hash`; the data card reports the realised episode mix by demand level and controller, the ambulance KPIs by controller and the label balance; the realised split counts are read from the `split` column (measured run: test 15, test_ood 12, train 79, val 14).
5. **Publish.** Compute the SHA-256 checksum of each table, the network table, the data dictionary and the configuration into a checksum file placed beside them (the file quoted in Section 12.4; the build step of this document writes it), and publish them with the data card at the download link that the report cites.

The 120-episode run used for the numbers in this document took 0.8–7.6 s per 4200-second episode (mean 2.3 s), so the 120-episode release plus its paired no-ambulance runs takes roughly 8 minutes on a laptop. The release size of 120 meets the proposal's commitment of at least 100 reproducible episodes with a margin, so that the 10% stress share yields about twelve out-of-distribution episodes for `test_ood`. File sizes of the measured run: episodes.csv 42 kB, vehicles.csv 83.2 MB, signal_ticks.csv 102.1 MB, junction_1hz.csv 52.4 MB.

12. Material for the Assignment 3 report, under the brief's section names

12.0 Front matter the report must carry

- **First page.** The brief asks whether the topic of Assignment 3 differs from Assignment 1. Answer: *No. The topic is unchanged; Sections 1–3 are revised from the proposal in response to the feedback and to the measured results (runtime, saturation flow, demand levels).*
- **Second page (3% of the unit mark).** Two headings, *Feedback received on Assignment 1* and *How we incorporated it* (at most 150 words), left as a placeholder until the feedback is released; the response should name the sections that changed and point to the evidence (for example the validation results of Section 10 or the measured KPIs of Section 8.4).
- **Files.** One PDF and one R Markdown file, never zipped; each member uploads the same report under their own name in the brief's naming pattern.

12.1 Characterising and Analysing Data (a): data sources and the four V's

- **Source.** Self-generated synthetic microsimulation, because vehicle-level data on ambulance passages cannot be collected safely on live roads and synthetic data from a validated simulator are the accepted substitute (proposal Section 1.1; Jordon et al., 2022). Version 1 is the model layer of the digital twin defined in the proposal (model, live data link, synchronisation loop; Kušić et al., 2023); no real road data enters it. Public data serve context only in version 1: the city congestion figures of the TomTom index (The Jakarta Post, 2024) and the city's open inventory of signals connected to its area traffic control system (Pemerintah Provinsi DKI Jakarta, n.d.). Calibration to a real Jakarta intersection is future work (Section 13): the signal inventory would give junction locations, open street geometry the block lengths, lane counts and speed limits, and the congestion figures a city-level delay to check a calibrated twin against. The demand levels of version 1 are calibrated from the simulator's own measured saturation flow (Section 9), not from public data. The dataset is the product: the twin's tables are the training data for the controller, the evidence for an investment decision, and a reusable benchmark.
- **Preprocessing.** The three steps the proposal names (warm-up filtering, episode-level splits, label post-pass) are built into the generator (Section 8.4) and are per episode, so preprocessing distributes trivially.
- **Volume.** The 120-episode run holds 300,598 decision rows, 360,443 vehicle rows and 1,996,932 one-second rows (237.8 MB of CSV). Rows scale linearly with episodes: a 120-episode release is about 301,000 decision rows and 238 MB; a thousand episodes would be a few million rows, still laptop-sized.
- **Velocity.** The physics runs at 2 Hz, the aggregates at 1 Hz, and a decision row is produced for each junction every 5 s of green. Generation throughput is about 910 logged episodes per hour including the paired no-ambulance runs (the logged runtime covers the logged episode only); in a live twin a decision must be served within the 5 s decision interval, and the ambulance channel updates at the rate of the fleet's GPS feed.
- **Variety.** Four relational tables at different grains (episode, vehicle, decision instant, second), a static network table, a JSON configuration, a data card; numeric, boolean, categorical and string (route) columns; optional trajectory logs for animation.
- **Veracity.** Ground truth is known by construction; realism is established by the validation experiments (saturation flow, Webster's 1958 delay term, the fundamental diagram); measurement noise is injected deliberately and documented; the limitations list (no amber, no lane changes, split phasing, Poisson arrivals, IDM drivers, no pedestrians, no motorcycles) is explicit. Synthetic is not real: results are about the twin.

12.2 Characterising and Analysing Data (b): platforms, software and tools, now and if the project expands

Role	Now (version 1, CSV only)	If the project expands
Generation and physics	Python 3.13 (3.11 or newer required), numpy, pandas; plain Python loop, one process per episode	SUMO (Lopez et al., 2018) through its control interface as an independently validated backend; a vectorised step if episodes ever exceed three minutes
Testing and validation	pytest (geometry, IDM, stop-or-commit, plant timing, invariants, determinism, labels, ambulances); the validation experiments of Section 10	the same tests against the SUMO backend
Storage and release	CSV tables, the configuration, the data dictionary, the data card and a checksum file; about 238 MB for 120 episodes	Parquet with DuckDB or Arrow for multi-gigabyte trajectory logs; object storage with a Zenodo DOI per release
Modelling	scikit-learn (Pedregosa et al., 2011): logistic regression, histogram gradient boosting, decision tree	distributed training (Spark or Dask) only beyond hundreds of millions of rows

Role	Now (version 1, CSV only)	If the project expands
Demonstration and report	R (R Core Team, 2026) with `dplyr`, `ggplot2`, `rpart` and `rmarkdown`; this document rendered from Markdown	unchanged
Versioning and provenance	`generator_version`, `config_hash` and the seed on every episode row; code under version control	versioned releases with DOIs

12.3 Characterising and Analysing Data (c): data analysis techniques and statistical methods, rationale and expected outcomes

- **How this realises proposal Goal 3.** The proposal promised a gradient-boosted model that predicts, per candidate phase, the ambulance delay and the general queue 30 s ahead, deployed as a priority-aware controller behind a safety shield. The dataset supports that goal in two supervised parts, not as a per-phase counterfactual (the design rules out re-running each step under each of the four phases). Task A imitates the priority-aware teacher, the rule that already trades the ambulance's arrival against the queue ahead of it (Section 7); Task B predicts the junction queue 30 s ahead *under the action that was logged*, and because the behaviour mixture of Section 6 applies different actions in similar states, the fitted regressor can be evaluated at each of the four candidate phases for a fixed state, which is the per-candidate-phase prediction the proposal describes, valid under the logged policies and reported as such. No ambulance-delay-ahead label is logged: ambulance delay exists after the fact, per vehicle in `vehicles.csv` and per episode in `episodes.csv`, and the ambulance effect of a phase choice is therefore measured in closed loop rather than predicted per row. A per-row ambulance-delay-ahead label is planned for the next generator version (Section 13). The deployed controller is the gradient-boosted model behind the plant of Section 5. Gradient-boosted trees are chosen because on tabular data of this size they match or beat deep models and stay inspectable (proposal Section 2.1; Grinsztajn et al., 2022), whereas deep reinforcement learning is hard to audit and is future work.
- **Descriptive.** Delay distributions by demand level and controller; label balance; feature–label correlations; the hidden aggressiveness α against individual delay (cars only: ambulances carry a fixed $\alpha = 1.0$); ambulance versus car delay; car delay by entry node under each controller (the fairness view below).
- **Task A, imitation (four classes).** Predict `y_mp_action` from the feature row. Logistic regression (class-balanced) as the interpretable baseline; a histogram gradient-boosting classifier as the main model; a depth-5 decision tree to show the recovered rule. Metrics: balanced accuracy, macro-F1, F1 of the "switch" rows, confusion matrix, permutation importance; truth features versus noisy sensor features.
- **Task B, per-candidate-phase regression.** Predict `y_q_total_t30` from the state and the *applied* action with Ridge and a histogram gradient-boosting regressor; report MAE, RMSE, R^2 and the partial dependence on the action, which is the control-relevant effect size (queue at $t + 30$ is mostly explained by queue at t, so R^2 alone flatters).
- **Validation design.** Grouped cross-validation by episode; the stress level as an out-of-distribution test; then, in the modelling stage, **closed-loop deployment** of the learned model as a controller (switch only if $P(\text{best}) > P(\text{current}) + 0.1$ for the classifier, or the phase with the lowest predicted cost for the regressor) on the evaluation grid of Section 7: fixed-time, max-pressure, naive full preemption, the teacher and the learned controller, at design and high demand, on at least eight paired seeds, scored on the primary and secondary metrics of Section 7. The teacher's own closed-loop result is the reference for the imitation, not a ceiling: an imperfect copy of a rule can do better or worse than the rule it copies.
- **Inference.** For a paired comparison over n seeds with differences Δ_i,

$$\overline{\Delta} \pm t_{n-1,\,0.975}\,\frac{s_\Delta}{\sqrt{n}},$$

with $n \geq 8$ ($t_{7,\,0.975} = 2.365$; never 1.96 at these sample sizes). A Wilcoxon signed-rank test cannot reach $p < 0.05$ with five pairs, so it is not used. This paired comparison is the evidence before capital spend that the proposal promises the city: the same ambulances and the same cars face each controller (common random numbers, Section 8.3), so the difference is the controller's.

- **The bound on general delay.** The proposal's acceptance criterion is that the learned controller keeps general-traffic delay within a stated margin of max-pressure. It is stated on cars, per demand level, as a paired difference $\Delta_i = \overline{D}_i^{\text{learned}} - \overline{D}_i^{\text{max-pressure}}$, where $\overline{D}_i$ is the episode's `mean_total_delay_s` (cars spawned in the main period) on seed i. The bound is met at a demand level when the upper end of the 95% t-interval on $\overline{\Delta}$ is at most the margin m. The margin is fixed before the evaluation runs and reported with the result; the group's proposed operating point is $m = 0.10\,\overline{D}^{\text{max-pressure}}$, ten per cent of the max-pressure mean car delay at that demand level, so that the bound scales with congestion. The full trade-off curve is reported in any case, so a city can move the operating point (proposal Section 2.4).
- **Fairness across approaches.** No episode-level column holds this; it is computed from `vehicles.csv` in the analysis. For each episode, take cars spawned in the main period, group them by `origin` (the eight entry nodes, each of which feeds one outer approach arm of one junction) and take the mean `total_delay_s` of each group, $\overline{D}_o$. The fairness statistic is the ratio of the largest to the smallest group mean, $F = \max_o \overline{D}_o / \min_o \overline{D}_o$ ($F = 1$ when every entry arm waits the same on average), reported per controller and demand level with the same paired-seed intervals as the other secondary metrics, and the eight $\overline{D}_o$ are shown as a bar chart so the reader sees *which* arm pays for the priority. Grouping by entry node attributes a trip's whole delay to its first approach; the per-approach queues `q_a` of `signal_ticks` give the complementary junction-level view.
- **Expected outcome, stated honestly.** Before the closed loop, the learned controller was expected, on car delay, to track max-pressure at design demand (its label is exactly the max-pressure request on every row without an ambulance), and, like its teacher, to lose to fixed time at high demand (Section 10, Experiment 5), where the max-pressure family leaves the grid over-saturated (Section 13), and to degrade further on stress. On ambulance travel time it is expected to approach the teacher it imitates; naive preemption may save more ambulance seconds still, at a larger cost to cars, which is the trade-off the report is about. The result is a table of the Section 7 metrics per controller and demand level with paired-seed intervals: ambulance seconds saved against the cost to cars relative to max-pressure and the bound above, the recovery time, and the fairness ratio. The operating point a city would choose is a threshold on that table, and reporting it cleanly is the result (proposal Section 2.4). In the pilot closed loop of Section 12.5 (same dataset), at design demand the learned controller's mean car delay is 91.3 s against 71.8 s for max-pressure (paired difference +19.5 s, 95% upper end +20.8 s, against a margin of 7.2 s; ratio 1.27, 95% interval 1.25 to 1.29), so the bound is not met; at high demand the learned controller's mean car delay is 338.7 s against 333.6 s for max-pressure (paired difference +5.1 s, 95% upper end +19.5 s, against a margin of 33.4 s; ratio 1.01, 95% interval 0.97 to 1.06), so the bound is met. Where the bound is not met, the levers for the modelling stage are the truth feature set, the hold threshold and a calmer teacher (Section 6), and the report states whichever outcome holds.

12.4 Characterising and Analysing Data (d): demonstration in R

The demonstration is the feasibility check the brief asks for and is kept separate from the analysis programme of 12.3: it characterises the dataset in R (tables, grain, features by namespace, missingness, basic distributions) and fits one small model to show that the label is learnable from the logged row. The modelling programme of 12.3 (gradient boosting, grouped cross-validation, closed-loop test, paired t-intervals) runs in Python and is reported under 12.3 and 12.5, not here. The demonstration is an R Markdown file submitted with the report. It reads the four relational CSV tables from the download link (and the network table for geometry), checks row counts and column names against the data dictionary, summarises missingness (labels are NaN only on tail and cool-down rows), and plots: the delay histogram by demand level; mean delay by controller; one junction's `q_total` over time; the label

balance; α against delay; ambulance versus car delay; mean car delay by entry node. One simple feasibility model (`rpart` on `y_mp_action`, or a logistic model of switch versus hold) is fitted with a by-episode hold-out. A minimal skeleton, run from the folder that holds the downloaded tables:

```
library(dplyr); library(ggplot2); library(rpart)
ep <- read.csv("episodes.csv")
vh <- read.csv("vehicles.csv")
st <- read.csv("signal_ticks.csv")
jh <- read.csv("junction_1hz.csv")
main <- st %>% filter(!is_warmup, !is_cooldown, !is_tail)
cars <- vh %>% filter(!is_warmup, vehicle_type == "car")   # alpha plots use cars
        only
ggplot(vh %>% filter(!is_warmup), aes(total_delay_s, fill = vehicle_type)) +
  geom_histogram(bins = 60, position = "identity", alpha = 0.6) + scale_x_log10()
train_ep <- ep$episode_id[ep$split == "train"]
fit <- rpart(factor(y_mp_action) ~ . - episode_id - junction_id - t - tick_idx
             - action_requested - action_applied - forced - switched -
        y_q_total_t30,
             data = main %>% filter(episode_id %in% train_ep), maxdepth = 5)
pred <- predict(fit, main %>% filter(!(episode_id %in% train_ep)), type = "class")
```

Release checklist (obligations from the brief). The report must carry a link to download the dataset, and the R Markdown file is uploaded as a separate file next to the PDF, named after the brief's pattern.

- **Download link.** *(download link to be inserted at release)* (generator traffic_twin_v1.2, configuration hash ecd36a4b3cad).
- **Integrity.** A checksum file is published beside the tables, and the report quotes the release checksums so that a reader can verify the download. Checksums of the 120-episode run (first 16 hexadecimal characters of each SHA-256; the checksum file holds the full values):

File	Size	SHA-256 (first 16)
`episodes.csv`	42 kB	`5ea49f6d4b8b8f30…`
`vehicles.csv`	83.2 MB	`2f20a4127ae9478d…`
`signal_ticks.csv`	102.1 MB	`e2454b2f83be6dc1…`
`junction_1hz.csv`	52.4 MB	`3c90f368f8fd8297…`
`network_edges.csv`	2 kB	`0f11ab4e1d22dd37…`
`data_dictionary.csv`	22 kB	`9edb99c68136024f…`
`config.json`	4 kB	`d03f7eab79b761f0…`

- **Licence.** CC BY 4.0 for the data and MIT for the code, stated in the data card and on the download page.

12.5 Results of the pilot models

Supervised models on the 120-episode release

Imitation of the priority-aware signal teacher (Task A), a 30 s-ahead queue forecast (Task B) and, in sections 4-6, the learned controller deployed in the simulator against the proposal's acceptance bound. Every number in this report is computed from the stored results of the same run.

1. Data

Release of 120 episodes (generator `traffic_twin_v1.2`, config hash `ecd36a4b3cad`): 253275 main-period decision rows from the signal-tick table (warm-up, cool-down and tail rows removed), split by episode: train 167542, val 30029, test 31390, test_ood 24314. Episodes per split: train 79, val 14, test

15, test_ood 12. Split integrity verified in code: every stress episode is test_ood and test_ood holds only stress episodes; random_dwell episodes are train-only; no episode is in two splits.

Scenario mixture (episodes): controllers max_pressure 48, max_pressure_eps 36, fixed_time 24, random_dwell 12; demand levels design 48, low 30, high 30, stress 12; sensor_noise_level 0.0 72, 0.1 48; ev_rate default 84, 0 36; demand_asym sym 60, ns_heavy 60; od_pattern default 84, uniform 36. Calibrated arrival rates (veh/h per entry node, mean over episodes): design 324.1, high 443.7, low 185.9, stress 522.0. Ambulance episodes 84 (2854 ambulances in total); gridlocked episodes 3; invariant violations 0. Label balance of `y_mp_action` on the main rows: N 27.4%, E 22.5%, S 27.8%, W 22.3%; the label differs from the current phase (a switch request) on 61.0% of rows; an ambulance is present on some approach in 20.2% of rows; `y_q_total_t30` is NaN on 0.01% of main rows (tail rows already removed).

Total runtime of this pipeline: 563 s (scikit-learn 1.7.2).

Feature sets (selected by namespace from the data dictionary)

- **truth** (63 columns): `phase`, `time_in_phase_s`, `max_green_remaining_s`, `q_N`, `n_N`, `q_N_l0`, `q_N_l1`, `arr_N_30s`, `q_E`, `n_E`, `q_E_l0`, `q_E_l1`, `arr_E_30s`, `q_S`, `n_S`, `q_S_l0`, `q_S_l1`, `arr_S_30s`, `q_W`, `n_W`, `q_W_l0`, `q_W_l1`, `arr_W_30s`, `down_n_N`, `down_occ_N`, `down_blocked_N`, `down_n_E`, `down_occ_E`, `down_blocked_E`, `down_n_S`, `down_occ_S`, `down_blocked_S`, `down_n_W`, `down_occ_W`, `down_blocked_W`, `nb_N_exists`, `nb_N_phase`, `nb_N_green_toward_me`, `nb_E_exists`, `nb_E_phase`, `nb_E_green_toward_me`, `nb_S_exists`, `nb_S_phase`, `nb_S_green_toward_me`, `nb_W_exists`, `nb_W_phase`, `nb_W_green_toward_me`, `ev_present_N`, `ev_dist_N`, `ev_eta_N`, `ev_q_ahead_N`, `ev_present_E`, `ev_dist_E`, `ev_eta_E`, `ev_q_ahead_E`, `ev_present_S`, `ev_dist_S`, `ev_eta_S`, `ev_q_ahead_S`, `ev_present_W`, `ev_dist_W`, `ev_eta_W`, `ev_q_ahead_W`
- **sensor** (47 columns): `phase`, `time_in_phase_s`, `max_green_remaining_s`, `obs_q_N`, `obs_q_E`, `obs_q_S`, `obs_q_W`, `down_n_N`, `down_occ_N`, `down_blocked_N`, `down_n_E`, `down_occ_E`, `down_blocked_E`, `down_n_S`, `down_occ_S`, `down_blocked_S`, `down_n_W`, `down_occ_W`, `down_blocked_W`, `nb_N_exists`, `nb_N_phase`, `nb_N_green_toward_me`, `nb_E_exists`, `nb_E_phase`, `nb_E_green_toward_me`, `nb_S_exists`, `nb_S_phase`, `nb_S_green_toward_me`, `nb_W_exists`, `nb_W_phase`, `nb_W_green_toward_me`, `ev_present_N`, `ev_dist_N`, `ev_eta_N`, `ev_q_ahead_N`, `ev_present_E`, `ev_dist_E`, `ev_eta_E`, `ev_q_ahead_E`, `ev_present_S`, `ev_dist_S`, `ev_eta_S`, `ev_q_ahead_S`, `ev_present_W`, `ev_dist_W`, `ev_eta_W`, `ev_q_ahead_W`
- **regression** (64 columns): `phase`, `time_in_phase_s`, `max_green_remaining_s`, `q_N`, `n_N`, `q_N_l0`, `q_N_l1`, `arr_N_30s`, `q_E`, `n_E`, `q_E_l0`, `q_E_l1`, `arr_E_30s`, `q_S`, `n_S`, `q_S_l0`, `q_S_l1`, `arr_S_30s`, `q_W`, `n_W`, `q_W_l0`, `q_W_l1`, `arr_W_30s`, `down_n_N`, `down_occ_N`, `down_blocked_N`, `down_n_E`, `down_occ_E`, `down_blocked_E`, `down_n_S`, `down_occ_S`, `down_blocked_S`, `down_n_W`, `down_occ_W`, `down_blocked_W`, `nb_N_exists`, `nb_N_phase`, `nb_N_green_toward_me`, `nb_E_exists`, `nb_E_phase`, `nb_E_green_toward_me`, `nb_S_exists`, `nb_S_phase`, `nb_S_green_toward_me`, `nb_W_exists`, `nb_W_phase`, `nb_W_green_toward_me`, `ev_present_N`, `ev_dist_N`, `ev_eta_N`, `ev_q_ahead_N`, `ev_present_E`, `ev_dist_E`, `ev_eta_E`, `ev_q_ahead_E`, `ev_present_S`, `ev_dist_S`, `ev_eta_S`, `ev_q_ahead_S`, `ev_present_W`, `ev_dist_W`, `ev_eta_W`, `ev_q_ahead_W`, `action_applied`

Never used as features (asserted in code): keys (`episode_id`, `junction_id`), time (`t`, `tick_idx`, period flags), `action_requested`, `forced`, `switched`, and the labels `y_*`. The sensor set keeps only what a loop-detector installation plus the ambulance GPS channel could deliver: no `n_*`, no per-lane queues, no arrival counts, `obs_q_*` instead of `q_*`. Note that the teacher rule is computed from `n_*` and `down_n_*`, so the sensor set can only approximate it.

2. Task A: imitation of the priority-aware teacher (4 classes, target `y_mp_action`)

Models: class-balanced logistic regression (one-hot phase columns + standardised numerics), HistGradientBoostingClassifier(max_iter=400, lr=0.05, depth=6, l2=1.0, early stopping, class-balanced) and a depth-5 decision tree. `switch_f1` is the F1 of the binary event *label != current phase* against *prediction != current phase*.

model	split	n	balanced_accuracy	macro_f1	switch_f1	accuracy
logreg_truth	val	30029	0.941	0.939	0.963	0.940
logreg_truth	test	31390	0.941	0.940	0.963	0.940
logreg_truth	test_ood	24314	0.919	0.918	0.960	0.919
hgb_truth	val	30029	0.959	0.958	0.977	0.959
hgb_truth	test	31390	0.949	0.948	0.970	0.949
hgb_truth	test_ood	24314	0.887	0.887	0.951	0.888
tree_truth	val	30029	0.472	0.457	0.750	0.478
tree_truth	test	31390	0.455	0.434	0.743	0.460
tree_truth	test_ood	24314	0.440	0.416	0.769	0.445
logreg_sensor	val	30029	0.736	0.735	0.787	0.737
logreg_sensor	test	31390	0.719	0.719	0.790	0.722
logreg_sensor	test_ood	24314	0.778	0.778	0.907	0.779
hgb_sensor	val	30029	0.768	0.767	0.843	0.769
hgb_sensor	test	31390	0.760	0.759	0.841	0.761
hgb_sensor	test_ood	24314	0.796	0.796	0.913	0.797
tree_sensor	val	30029	0.456	0.374	0.708	0.408
tree_sensor	test	31390	0.418	0.351	0.701	0.384
tree_sensor	test_ood	24314	0.476	0.444	0.774	0.458

GroupKFold(5, groups = episode_id) on the train split, HGB

model	balanced_accuracy	macro_f1	folds
hgb_truth	0.948 +- 0.010	0.947 +- 0.010	0.945, 0.959, 0.933, 0.959, 0.942
hgb_sensor	0.770 +- 0.011	0.768 +- 0.011	0.767, 0.766, 0.791, 0.762, 0.762

Per-class F1 on the test split

model	F1_N	F1_E	F1_S	F1_W	support_N	support_E	support_S	support_W
logreg_truth	0.944	0.935	0.945	0.936	8551	6830	9128	6881
hgb_truth	0.949	0.946	0.954	0.945	8551	6830	9128	6881
tree_truth	0.549	0.436	0.442	0.309	8551	6830	9128	6881
logreg_sensor	0.731	0.701	0.745	0.699	8551	6830	9128	6881
hgb_sensor	0.769	0.739	0.782	0.747	8551	6830	9128	6881
tree_sensor	0.268	0.446	0.242	0.449	8551	6830	9128	6881

Confusion matrices

Figure: Task A confusion matrices (counts and row shares) for every model and split. HGB-truth on test:

label	pred_N	pred_E	pred_S	pred_W
N	8093	154	169	135
E	117	6489	123	101
S	149	132	8700	147
W	143	120	116	6502

HGB-sensor on test:

label	pred_N	pred_E	pred_S	pred_W
N	6525	618	766	642

label	pred_N	pred_E	pred_S	pred_W
E	629	5063	603	535
S	712	653	7121	642
W	557	538	598	5188

Permutation importance, HGB-sensor, test split (20000 rows, 5 repeats, balanced accuracy)

feature	mean	std
phase	0.2585	0.0033
obs_q_S	0.1753	0.0015
obs_q_N	0.1630	0.0013
obs_q_E	0.1338	0.0021
obs_q_W	0.1273	0.0023
ev_present_W	0.0621	0.0010
ev_present_E	0.0584	0.0012
ev_present_N	0.0507	0.0008
ev_present_S	0.0473	0.0006
down_n_W	0.0410	0.0012
down_n_E	0.0314	0.0012
down_n_N	0.0231	0.0011
down_n_S	0.0195	0.0020
time_in_phase_s	0.0162	0.0020
down_occ_N	0.0133	0.0014

Figure: permutation importance of the HGB-sensor model on the test split.

Depth-5 decision tree (first three levels, truth features)

```
|--- ev_dist_W <= -0.50
|   |--- ev_eta_E <= -0.50
|   |   |--- ev_q_ahead_N <= -0.50
|   |   |   |--- n_S <= 5.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- n_S >  5.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |--- ev_q_ahead_N >  -0.50
|   |   |   |--- ev_present_S <= 0.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_present_S >  0.50
|   |   |   |   |--- truncated branch of depth 2
|   |--- ev_eta_E >  -0.50
|   |   |--- ev_eta_N <= -0.50
|   |   |   |--- ev_eta_S <= -0.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_eta_S >  -0.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |--- ev_eta_N >  -0.50
|   |   |   |--- ev_dist_N <= 21.44
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_dist_N >  21.44
|   |   |   |   |--- truncated branch of depth 2
|--- ev_dist_W >  -0.50
|   |--- ev_dist_N <= -0.50
```

```
|   |   |--- ev_dist_E <= -0.50
|   |   |   |--- ev_eta_S <= -0.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_eta_S >  -0.50
|   |   |   |   |--- truncated branch of depth 2
|   |   |--- ev_dist_E >  -0.50
|   |   |   |--- ev_dist_E <= 14.97
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_dist_E >  14.97
|   |   |   |   |--- truncated branch of depth 2
|   |--- ev_dist_N >  -0.50
|   |   |--- ev_eta_W <= 5.33
|   |   |   |--- ev_eta_N <= 2.82
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_eta_N >  2.82
|   |   |   |   |--- truncated branch of depth 2
|   |   |--- ev_eta_W >  5.33
|   |   |   |--- ev_dist_N <= 34.41
|   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- ev_dist_N >  34.41
|   |   |   |   |--- truncated branch of depth 2
```

Full rules: `taskA_tree_rules_truth.txt`, `taskA_tree_rules_sensor.txt`.

3. Task B: queue 30 s ahead (target `y_q_total_t30`, feature set `regression` = truth + action_applied)

model	split	n	mae	rmse	r2	target_mean
naive: q_total now	val	30029	3.40	4.65	0.978	23.07
ridge	val	30029	2.91	3.85	0.985	23.07
hgb	val	30029	2.41	3.22	0.989	23.07
naive: q_total now	test	31390	3.68	5.29	0.973	27.36
ridge	test	31390	3.09	4.23	0.983	27.36
hgb	test	31390	2.56	3.41	0.989	27.36
naive: q_total now	test_ood	24314	5.92	7.61	0.928	97.08
ridge	test_ood	24314	4.79	6.07	0.954	97.08
hgb	test_ood	24314	4.22	5.36	0.964	97.08

Partial dependence of the HGB prediction on `action_applied` (31390 test rows)

action_name	mean_pred_queue	mean_pred_queue_when_hold	mean_pred_queue_when_switch	n_rows
N	27.37	27.71	27.25	31390
E	27.72	28.06	27.61	31390
S	27.33	27.65	27.22	31390
W	27.93	26.66	28.34	31390

Holding the current phase gives a mean predicted queue of 27.52 veh; the best other action 26.91 veh; in 69.8% of test rows some switch is predicted to lower the queue at t+30. Figure: partial dependence of the Task B prediction on the applied action.

The 30 s-ahead queue is mostly explained by the queue now (see the naive baseline), so R^2 alone flatters; the control-relevant quantity is the action effect above, which is small compared with the state effect.

Reading Tasks A and B

- The teacher is a deterministic rule of `n_*`, `down_n_*`, `down_blocked_*`, `time_in_phase_s` and the `ev_*` channel. With truth features HGB reaches balanced accuracy 0.949 on test and 0.887 on stress (test_ood); the remaining errors sit on the hysteresis boundary (pressure difference close to 2) and on max-green rows.
- With sensor features (queues only, noisy in the noisy episodes) HGB reaches 0.760 on test and 0.796 on stress: the gap to the truth model is the price of not observing moving vehicles (`n_a`) and arrivals.
- Logistic regression (truth) reaches 0.941: the rule is a difference of counts with a threshold, which a linear model can only partly express across phases.
- Task B: HGB MAE 2.56 veh vs naive 3.68 veh on test; on stress 4.22 vs 5.92.

4. Closed loop: the learned controller deployed in the simulator

Protocol: controllers ['fixed_time', 'max_pressure', 'preempt_naive', 'priority_mp', 'ml_sensor'] x demand ['design', 'high'] x 8 paired seeds (900001..900008, disjoint from the training seeds 100000..100119), demand_asym sym, od default, ev_rate default (lambda_ev = 4.0 /h per node), sensor_noise_level 0. Every (controller, demand, seed) was also run with ambulances off (same seed; 112 such episodes) to measure `car_mean_delay_penalty_s`. 224 episodes in total, wall clock 352 s with 8 worker processes (serial inference per worker, BLAS/OMP threads = 1). The ML controller is the gradient-boosted-tree classifier on sensor features, called `ml_sensor` in the tables, with the hysteresis rule request argmax only if P(best) > P(current) + 0.10. Invariant violations over all episodes: 0; gridlocked episodes: 0.

Simulator `traffic_twin_v1.2`, config hash `ecd36a4b3cad`; dataset `traffic_twin_v1.2`, config hash `ecd36a4b3cad` (asserted equal: True).

4.1 Mean over seeds per controller and demand level (ambulances on)

demand	controller	n	ev_mean_travel_time_s	ev_mean_delay_s	ev_p90_delay_s	car_mean_delay_s	car_p90_delay_s	throughput_vph	switches_per_min	gridlock_rate	n_red_commits	n_ambulances
design	fixed_time	8	123.78	91.51	146.21	100.48	156.61	2622	2.50	0.00	287	34.0
design	max_pressure	8	100.49	68.23	109.78	71.83	123.09	2609	4.22	0.00	755	34.0
design	preempt_naive	8	50.85	18.58	28.50	73.19	126.19	2608	4.19	0.00	756	34.0
design	priority_mp	8	56.93	24.69	40.31	73.47	125.76	2607	4.21	0.00	754	34.0
design	ml_sensor	8	56.30	24.05	37.41	91.31	148.04	2603	3.84	0.00	668	34.0
high	fixed_time	8	192.40	160.22	249.47	155.41	254.47	3520	2.50	0.00	429	34.0
high	max_pressure	8	335.76	309.97	491.54	333.56	550.76	3147	4.41	0.00	1240	34.0
high	preempt_naive	8	131.67	102.94	219.18	282.14	458.77	3252	4.09	0.00	1132	34.0
high	priority_mp	8	192.62	178.02	310.56	367.71	621.10	3091	4.68	0.00	1459	34.0
high	ml_sensor	8	160.87	144.04	270.41	338.68	568.46	3164	4.27	0.00	1184	34.0

4.2 The proposal's acceptance bound: car mean delay under the ML controller <= 1.10 x `max_pressure`

Per seed the ratio car_delay(ml_sensor) / car_delay(max_pressure) is computed on the paired episodes; PASS iff ratio <= 1.10. `mean_ratio` carries a t-based 95% interval over seeds; `bound_met_ci` = that interval's upper end is below the factor. The proposal states the same bound on the paired difference: `bound_met_methodology` = the upper end of the 95% t-interval on mean(ml - max_pressure) is at most `margin_s` = 0.10 x the mean max_pressure car delay of that cell.

sensor_noise_level	demand_level	n_pairs	n_pass	pass_rate	mean_ratio	ci95_low	ci95_high	min_ratio	max_ratio	mean_ml	mean_reference	bound_met_all_seeds	bound_met_ci
0	design	8	0	0.000	1.272	1.251	1.293	1.241	1.320	91.3	71.8	False	False
0	high	8	8	1.000	1.014	0.971	1.056	0.948	1.097	338.7	333.6	True	True
0	all	16	8	0.500	1.143	1.069	1.217	0.948	1.320	215.0	202.7	False	False
0.1	design	8	0	0.000	1.324	1.299	1.349	1.276	1.362	95.1	71.8	False	False
0.1	high	8	8	1.000	0.954	0.906	1.002	0.872	1.042	317.8	333.6	True	True
0.1	all	16	8	0.500	1.139	1.034	1.243	0.872	1.362	206.5	202.7	False	False

The same cells in the methodology's paired-difference form (seconds of car delay):

sensor_noise_level	demand_level	n_pairs	diff_mean_s	diff_ci95_half_s	diff_ci95_high_s	margin_s	bound_met_methodology
0	design	8	19.48	1.28	20.77	7.18	False
0	high	8	5.11	14.43	19.54	33.36	True
0	all	16	12.30	7.44	19.74	20.27	True
0.1	design	8	23.28	2.17	25.44	7.18	False
0.1	high	8	-15.72	17.32	1.60	33.36	True
0.1	all	16	3.78	13.15	16.93	20.27	True

Figure: car-delay ratio of the ML controller to the reference per demand level, with the 1.10 bound.

4.3 Paired differences versus `max_pressure` (mean +- t-based 95% half-width, n = pairs)

Negative delay / penalty / recovery / fairness differences favour the controller; positive throughput differences favour it. ∗ = the 95% interval excludes zero.

design demand

controller	ev_mean_travel_time_s	ev_mean_delay_s	mean_total_delay_s	p90_total_delay_s	throughput_vph	switches_per_min	n_red_commits	gridlock
fixed_time	+23.29 +- 6.47 (n=8) *	+23.29 +- 6.48 (n=8) *	+28.65 +- 1.58 (n=8) *	+33.51 +- 2.47 (n=8) *	+13.00 +- 9.50 (n=8) *	-1.72 +- 0.02 (n=8) *	-468.12 +- 30.08 (n=8) *	+0.00 +- 0.00 (n=8)
preempt_naive	-49.64 +- 5.02 (n=8) *	-49.64 +- 5.01 (n=8) *	+1.36 +- 1.44 (n=8)	+3.10 +- 3.70 (n=8)	-1.38 +- 4.47 (n=8)	-0.03 +- 0.05 (n=8)	+0.50 +- 25.74 (n=8)	+0.00 +- 0.00 (n=8)
priority_mp	-43.56 +- 4.57 (n=8) *	-43.54 +- 4.58 (n=8) *	+1.64 +- 1.21 (n=8) *	+2.67 +- 2.30 (n=8) *	-2.12 +- 4.20 (n=8)	-0.00 +- 0.05 (n=8)	-1.00 +- 26.36 (n=8)	+0.00 +- 0.00 (n=8)
ml_sensor	-44.19 +- 4.60 (n=8) *	-44.18 +- 4.60 (n=8) *	+19.48 +- 1.28 (n=8) *	+24.95 +- 2.30 (n=8) *	-5.50 +- 5.38 (n=8) *	-0.38 +- 0.06 (n=8) *	-87.12 +- 25.61 (n=8) *	+0.00 +- 0.00 (n=8)

high demand

controller	ev_mean_travel_time_s	ev_mean_delay_s	mean_total_delay_s	p90_total_delay_s	throughput_vph	switches_per_min	n_red_commits	gridlock
fixed_time	-143.36 +- 32.61 (n=8) *	-149.75 +- 34.47 (n=8) *	-178.15 +- 25.21 (n=8) *	-296.29 +- 34.83 (n=8) *	+372.88 +- 40.33 (n=8) *	-1.91 +- 0.02 (n=8) *	-810.62 +- 33.72 (n=8) *	+0.00 +- 0.00 (n=8)
preempt_naive	-204.08 +- 30.26 (n=8) *	-207.03 +- 29.50 (n=8) *	-51.42 +- 19.77 (n=8) *	-91.99 +- 27.78 (n=8) *	+104.62 +- 21.31 (n=8) *	-0.32 +- 0.06 (n=8) *	-107.12 +- 41.66 (n=8) *	+0.00 +- 0.00 (n=8)
priority_mp	-143.14 +- 26.32 (n=8) *	-131.95 +- 17.93 (n=8) *	+34.15 +- 11.57 (n=8) *	+70.34 +- 30.54 (n=8) *	-56.62 +- 23.97 (n=8) *	+0.27 +- 0.07 (n=8) *	+219.12 +- 47.20 (n=8) *	+0.00 +- 0.00 (n=8)
ml_sensor	-174.89 +- 25.65 (n=8) *	-165.93 +- 21.17 (n=8) *	+5.11 +- 14.43 (n=8)	+17.70 +- 33.54 (n=8)	+16.50 +- 25.00 (n=8)	-0.14 +- 0.08 (n=8) *	-55.50 +- 32.62 (n=8) *	+0.00 +- 0.00 (n=8)

4.4 Secondary metrics (ambulance, car penalty, recovery, fairness)

`ev_mean_travel_time_s` = mean ambulance travel time; `car_mean_delay_penalty_s` = car mean delay with ambulances minus the paired episode without ambulances (same seed and controller); `recovery_time_s` = mean time after an ambulance clears a junction until that junction's queue is back within 10% of its value at first sighting; `fairness_ratio` = max / min of the mean car delay over the 8 entry nodes (1 = perfectly even).

demand	controller	ev_mean_travel_time_s	car_mean_delay_penalty_s	recovery_time_s	fairness_ratio	worst_origin (mode)
design	fixed_time	123.78	0.09	82.28	1.358	E1
design	max_pressure	100.49	2.04	54.64	1.128	E1
design	preempt_naive	50.85	3.40	58.93	1.158	E0
design	priority_mp	56.93	3.68	62.54	1.128	S0
design	ml_sensor	56.30	1.86	60.44	1.127	E1
high	fixed_time	192.40	3.65	69.20	1.322	N0
high	max_pressure	335.76	15.80	126.58	1.208	E1
high	preempt_naive	131.67	-35.62	46.15	1.195	N1
high	priority_mp	192.62	49.95	93.93	1.356	N0
high	ml_sensor	160.87	23.89	62.03	1.369	W1

Paired differences of the secondary metrics versus `max_pressure`:

design demand

controller	ev_mean_travel_time_s	car_mean_delay_penalty_s	recovery_time_s	fairness_ratio
fixed_time	+23.29 +- 6.47 (n=8) *	-1.95 +- 1.80 (n=8) *	+27.64 +- 39.26 (n=8)	+0.230 +- 0.053 (n=8) *
preempt_naive	-49.64 +- 5.02 (n=8) *	+1.36 +- 1.44 (n=8)	+4.30 +- 23.60 (n=8)	+0.030 +- 0.040 (n=8)
priority_mp	-43.56 +- 4.57 (n=8) *	+1.64 +- 1.21 (n=8) *	+7.90 +- 32.98 (n=8)	+0.000 +- 0.036 (n=8)
ml_sensor	-44.19 +- 4.60 (n=8) *	-0.18 +- 3.45 (n=8)	+5.80 +- 32.25 (n=8)	-0.001 +- 0.032 (n=8)

high demand

controller	ev_mean_travel_time_s	car_mean_delay_penalty_s	recovery_time_s	fairness_ratio
fixed_time	-143.36 +- 32.61 (n=8) *	-12.15 +- 10.47 (n=8) *	-57.38 +- 93.26 (n=8)	+0.114 +- 0.125 (n=8)
preempt_naive	-204.08 +- 30.26 (n=8) *	-51.42 +- 19.77 (n=8) *	-80.43 +- 41.66 (n=8) *	-0.013 +- 0.097 (n=8)
priority_mp	-143.14 +- 26.32 (n=8) *	+34.15 +- 11.57 (n=8) *	-32.65 +- 52.95 (n=8)	+0.148 +- 0.131 (n=8) *
ml_sensor	-174.89 +- 25.65 (n=8) *	+8.09 +- 16.39 (n=8)	-64.55 +- 47.06 (n=8) *	+0.160 +- 0.211 (n=8)

4.5 Sensor-noise variant (the sensor-realistic claim)

`ml_sensor` and `max_pressure` re-run on the same 8 seeds x ['design', 'high'] at sensor_noise_level 0.1 (obs_q_a = Binomial(q_a, 1-p_miss) + Poisson(p_miss/2)); the ML controller reads `obs_q_*`, max_pressure reads the true `n_*` and is therefore unaffected by construction except through the ambulance stream, which is identical. Means over seeds:

demand	controller	n	car_delay_noise0	car_delay_noise	ev_delay_noise0	ev_delay_noise	throughput_noise0	throughput_noise	agreement_noise0	agreement_noise	switches_noise
design	max_pressure	8	71.83	71.83	68.23	68.23	2609	2609	0.900	0.900	4.22
design	ml_sensor	8	91.31	95.10	24.05	25.09	2603	2598	0.674	0.652	3.77
high	max_pressure	8	333.56	333.56	309.97	309.97	3147	3147	0.695	0.695	4.41
high	ml_sensor	8	338.68	317.84	144.04	121.52	3164	3212	0.707	0.682	4.15

Paired differences `ml_sensor` - `max_pressure` at noise 0.1:

demand	mean_total_delay_s	ev_mean_delay_s	throughput_vph	car_mean_delay_penalty_s	fairness_ratio
design	+23.28 +- 2.17 (n=8) *	-43.14 +- 4.00 (n=8) *	-10.38 +- 9.99 (n=8) *	+0.13 +- 3.32 (n=8)	+0.013 +- 0.030 (n=8)
high	-15.72 +- 17.32 (n=8)	-188.45 +- 23.83 (n=8) *	+64.25 +- 27.51 (n=8) *	+12.82 +- 17.49 (n=8)	+0.109 +- 0.205 (n=8)

The acceptance bound at this noise level is in the table of 4.2 (rows with sensor_noise_level 0.1).

4.6 Request statistics on main-period decision rows (per controller, mean over episodes, noise 0)

`agreement_with_teacher` = share of decision instants where the controller's request equals the priority-aware teacher label recomputed on the same row (for `priority_mp` it is 1 by construction); `request_switch_rate` = share of requests that differ from the current phase; `accepted_switch_rate` = share of instants at which the plant actually started an all-red; `forced_share` = share of instants where the plant overrode the request (max-green or starvation guard).

demand	controller	agreement_with_teacher	disagreement_rate	request_switch_rate	teacher_switch_rate	accepted_switch_rate	forced_share	n_decisions_main
design	fixed_time	0.370	0.630	0.250	0.711	0.250	0.000	2395
design	max_pressure	0.900	0.100	0.490	0.535	0.490	0.000	2065
design	preempt_naive	0.987	0.013	0.485	0.485	0.485	0.000	2071
design	priority_mp	1.000	0.000	0.490	0.490	0.490	0.000	2066
design	ml_sensor	0.674	0.326	0.431	0.574	0.431	0.001	2138
high	fixed_time	0.322	0.678	0.250	0.738	0.250	0.000	2396
high	max_pressure	0.695	0.305	0.522	0.671	0.522	0.000	2027
high	preempt_naive	0.917	0.083	0.469	0.533	0.470	0.002	2089
high	priority_mp	1.000	0.000	0.567	0.567	0.569	0.004	1975
high	ml_sensor	0.707	0.293	0.498	0.593	0.499	0.003	2055

Figure: ambulance and car delay per controller and demand level, with paired differences to the reference. The per-episode rows (ambulance-off and noise-variant episodes included), the paired differences, the bound and the means per cell are published as tables with this report.

5. Verdict

- **Imitation (Task A).** HGB-truth reaches balanced accuracy 0.949 on test and 0.887 on stress (test_ood); HGB-sensor, the deployed model, 0.760 / 0.796 (switch F1 0.841 on test). The gap between the two is the price of not observing moving vehicles and arrivals; the teacher is a rule of `n_*`, which no loop detector delivers.
- **Queue forecast (Task B).** HGB MAE 2.56 veh on test vs 3.68 veh for the naive 'queue now' baseline (R^2 0.989); on stress 4.22 vs 5.92. The action effect in the partial dependence is -0.61 veh (best other action minus hold), small against the state effect.
- **Acceptance bound, design, noise 0**: 0/8 seeds pass (pass rate 0.00); mean ratio 1.272 [1.251, 1.293] (car delay 91.3 s vs 71.8 s). The bound is NOT met on every seed. Methodology form: paired difference +19.5 s, 95% upper end +20.8 s against a margin of 7.2 s: NOT met.
- **Acceptance bound, high, noise 0**: 8/8 seeds pass (pass rate 1.00); mean ratio 1.014 [0.971, 1.056] (car delay 338.7 s vs 333.6 s). The bound is met on every seed and the interval lies below 1.10. Methodology form: paired difference +5.1 s, 95% upper end +19.5 s against a margin of 33.4 s: met.
- **Acceptance bound, pooled over demand levels, noise 0**: pass rate 0.50 (8/16), mean ratio 1.143 [1.069, 1.217].
- **Acceptance bound, design, noise 0.1**: 0/8 seeds pass, mean ratio 1.324 [1.299, 1.349].
- **Acceptance bound, high, noise 0.1**: 8/8 seeds pass, mean ratio 0.954 [0.906, 1.002].
- **design**: on ambulance delay the ML controller beats max_pressure (-44.2 +- 4.6 s); on car delay it loses to max_pressure (+19.5 +- 1.3 s). Against its own teacher priority_mp the ML controller's ambulance delay differs by -0.6 s (mean over seeds). The teacher itself changes ambulance delay by -43.5 +- 4.6 s and naive preemption by -49.6 +- 5.0 s versus max_pressure.

    - car delay penalty of the ambulances: ML does not differ significantly from max_pressure (-0.178 +- 3.447; ML 1.858 vs 2.036).
    - recovery time: ML does not differ significantly from max_pressure (+5.801 +- 32.253; ML 60.440 vs 54.639).
    - fairness ratio: ML does not differ significantly from max_pressure (-0.001 +- 0.032; ML 1.127 vs 1.128).
- **high**: on ambulance delay the ML controller beats max_pressure (-165.9 +- 21.2 s); on car delay it does not differ significantly from max_pressure (+5.1 +- 14.4 s). Against its own teacher priority_mp the ML controller's ambulance delay differs by -34.0 s (mean over seeds). The teacher itself changes ambulance delay by -131.9 +- 17.9 s and naive preemption by -207.0 +- 29.5 s versus max_pressure.
    - car delay penalty of the ambulances: ML does not differ significantly from max_pressure (+8.088 +- 16.386; ML 23.890 vs 15.801).
    - recovery time: ML beats max_pressure (-64.549 +- 47.062; ML 62.031 vs 126.580).
    - fairness ratio: ML does not differ significantly from max_pressure (+0.160 +- 0.211; ML 1.369 vs 1.208).
- **Reference check, high**: plain fixed_time beats max_pressure on car delay (-178.1 +- 25.2 s, throughput +373 +- 40 veh/h). The max-pressure rule as specified for this simulator (split phasing, one-approach pressure, 2-vehicle hysteresis) is therefore not a strong baseline at this demand; the ML controller imitates a teacher built on that rule and inherits the weakness. This is a finding about the simulator's controllers, reported as is.
- **Agreement.** Deployed, the ML controller's request equals the teacher's on 69.0% of main-period decision instants (mean over its episodes at noise 0); the residual disagreement is concentrated on hysteresis-boundary and max-green rows and on ambulance rows, where the sensor set cannot see `n_*`.
- **Safety.** 0 invariant violations in 224 closed-loop episodes: the signal plant, not the model, guarantees safety.
- **Scope.** All of this is one simulator with its own teacher rule and calibrated demand; the closed-loop seeds are disjoint from training but the demand model, OD pattern and vehicle population are the same as in training. Nothing here is evidence about a real junction.

6. Reproducibility

- Simulator: `traffic_twin_v1.2`, config hash `ecd36a4b3cad`; dataset `traffic_twin_v1.2`, config hash `ecd36a4b3cad`, 120 episodes.
- Software at evaluation time: Python 3.13.2, numpy 2.2.6, pandas 2.3.3, scikit-learn 1.7.2, joblib 1.5.2; macOS-15.2-arm64-arm-64bit-Mach-O.
- Supervised pipeline: 563 s; scikit-learn 1.7.2, dataset config hash `ecd36a4b3cad`.
- Closed loop: 352 s, 8 workers, seeds 900001..900008, hysteresis margin 0.1, noise variant 0.1.
- Every number in this report is computed from the stored per-episode results; the report can be rebuilt without re-simulating.

12.6 Standard for Data Science Process, Data Governance and Management

- **Process standard.** The CRISP-DM phases (Chapman et al., 2000) map onto the milestones: business understanding (proposal), data understanding (validation and data card), data preparation (generator, flags, splits), modelling (tasks A and B), evaluation (closed loop on paired seeds), deployment (offline replay of a logged episode, the first digital-twin step of Section 13). Code follows PEP 8 (van Rossum et al., 2001) with unit tests and a semantic `generator_version`.

- **Data standards.** FAIR (Wilkinson et al., 2016): findable (download link, data card), accessible (CSV), interoperable (SI units, data dictionary with namespaces), reusable (licence, configuration and seed regenerate any row). The data card follows the datasheets-for-datasets pattern (Gebru et al., 2021). Quality dimensions: completeness (conservation invariant), consistency (label recompute test), accuracy (validation experiments).
- **Accessibility.** Public download link (12.4), CSV readable by any tool, data dictionary and data card alongside, versioned releases named by `generator_version` and `config_hash`; any row can be regenerated from the configuration and the seed, so the data never depend on one hosting location.
- **Security and integrity.** SHA-256 checksums published with the link; code and data under version control; a corrupted or altered copy is detected by the checksum and rebuilt from the seed rather than trusted. Retention as versioned releases.
- **Confidentiality.** The dataset is synthetic: no personal data, no consent issues, no re-identification risk. Provenance is (generator version, configuration hash, seed). Proposed licences: CC BY 4.0 for data, MIT for code.
- **What changes with real data.** A deployed system would ingest ambulance GPS traces, which are location data adjacent to patient information: health-data regulation, de-identification, access control and retention limits would apply before any real feed enters the twin. The signal side (queue counts from the control centre's cameras) is operational data without personal content.
- **Ethics and safety.** This is a research artefact. The learned rule is designed to be deployable behind an existing signal controller's interlocks, as the proposal says, but nothing here drives a real signal: the safety shield (the plant of Section 5) is simulated, and a deployment would first need the real-data steps above. Priority shifts delay onto general traffic, so fairness across approaches is measured as the spread of mean car delay across the eight entry nodes (12.3) and reported next to the ambulance gain, so that the trade-off the proposal promises is a number, not a sentence; the plant's starvation guard (Section 5), not the max-green rule, is what stops an approach starving: whatever the controller requests, an occupied approach gets a green within the 180 s guard plus one decision interval and the all-reds around it, and `max_approach_red_s` records the realised worst case per episode. The label rule is a published, inspectable formula. Generative-AI use is disclosed in 12.7, as the unit requires.

12.7 Use of generative AI (statement for the report)

The proposal's statement is carried forward and widened to cover what has been produced since. Wording for the report, to be checked by every group member before submission:

"We used Claude (Anthropic, via Claude Code, September–November 2026) to brainstorm candidate framings, to critique our simulator design, to draft the simulator code, its unit tests, the dataset generator, the validation and modelling scripts, the digital-twin extension, the R Markdown demonstration and this methodology companion, all of which we reviewed, ran, tested and modified, to draft and revise the proposal and this report from our problem choice, design decisions and notes, and to check reference details by web search. The choice of problem, the design decisions and the cited sources are our own, and we re-checked every reference against its published version."

13. Limitations and future work

No amber (replaced by stop-or-commit and an extended all-red); no lane changing (merge inside the box); split phasing, which lowers capacity relative to paired phasing; Poisson arrivals with a flat profile; IDM drivers without reaction time; no pedestrians or motorcycles; a box-empty clearance gate rather than a per-movement conflict matrix; free-flow time ignoring the turn slow-down; the box counted empty when a vehicle's front bumper leaves it; no teacher-driven episodes in the behaviour mixture; no per-row ambulance-delay-ahead label.

Known properties of the specified rules, documented in the data card: the max-pressure teacher switches about four times a minute and loses to fixed time at high demand; hard braking near the stop-or-commit boundary occurs at 0.63 events per red onset in the design episode (Section 10, Experiment 3); the realised junction capacity is about 10% below the textbook formula, which is why demand is calibrated against the measured value; and the starvation guard bounds the wait for a green, not the wait of a lane head behind an exit lane blocked by spillback, which the liveness measurements of Section 4.3 record.

The demand calibration is single-junction, and on the 2×2 grid the nominal high level is over capacity for the max-pressure family: 11 of 17 high-demand max-pressure and ε-greedy episodes end with more than 380 vehicles in the network (median 399, maximum 512), against a mean of 68 (maximum 198) under fixed time at the same level. The learned controller, trained on that teacher, inherits the property, so its high-demand results describe an over-saturated grid.

After the release, a first lock-step twin was built on top of the dataset as an extension: a hidden 'physical' run of the simulator feeds noisy, delayed detectors (loops, a queue camera per edge with misses, dropped frames and a 1 s delivery delay); a state estimator rebuilds the same feature row as the dataset from those detectors alone; the trained sensor-feature model controls the lights from that row; and a drift monitor compares estimate and detectors. Over a 24-hour simulated day with the daily demand profile at design level it handled 45,416 vehicles with 0 gridlocks and 0 invariant violations, a mean queue-estimation error of 0.64 vehicles, and 27 stuck-camera events detected and recovered. An offline replay of a logged showcase episode through the same estimator tracks the true queues with a mean absolute error of 0.60 vehicles per edge. The ambulance channel is read without noise in the twin, as in the dataset.

Planned: the per-row ambulance-delay-ahead label (the extra seconds the nearest ambulance on an approach spends queued over the next 30 s, computed in the post-pass from a per-ambulance 1 Hz log); a live link to real detectors and ambulance GPS, the remaining part of a digital twin in the sense of Kušić et al. (2023); SUMO (Lopez et al., 2018) as an external validation backend; reinforcement learning after the course (Wei et al., 2019); GPS noise and delivery delay on the ambulance channel; and calibration to a real Jakarta intersection.

References

Brice, S. N., Boutilier, J. J., Gartner, D., Harper, P., Knight, V., Lloyd, J., Pusponegoro, A. D., Rini, A. P., Turnbull-Ross, J., & Tuson, M. (2022). Emergency services utilization in Jakarta (Indonesia): A cross-sectional study of patients attending hospital emergency departments. *BMC Health Services Research, 22*, Article 639. https://doi.org/10.1186/s12913-022-08061-8

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.

Chen, Y.-Y., & Wang, J.-Y. (2024). An emergency vehicle traffic signal preemption system considering queue spillbacks along routes and negative impacts on non-priority traffic. *IET Intelligent Transport Systems, 18*(8), 1385–1395. https://doi.org/10.1049/itr2.12518

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé, H., III, & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM, 64*(12), 86–92. https://doi.org/10.1145/3458723

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35* (Datasets and Benchmarks Track). https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html

Jordon, J., Szpruch, L., Houssiau, F., Bottarelli, M., Cherubin, G., Maple, C., Cohen, S. N., & Weller, A. (2022). *Synthetic data – what, why and how?* arXiv. https://doi.org/10.48550/arXiv.2205.03257

Kušić, K., Schumann, R., & Ivanjko, E. (2023). A digital twin in transportation: Real-time synergy of traffic data streams and simulation for virtualizing motorway dynamics. *Advanced Engineering Informatics, 55*, Article 101858. https://doi.org/10.1016/j.aei.2022.101858

Larsen, M. P., Eisenberg, M. S., Cummins, R. O., & Hallstrom, A. P. (1993). Predicting survival from out-of-hospital cardiac arrest: A graphic model. *Annals of Emergency Medicine, 22*(11), 1652–1658. https://doi.org/10.1016/S0196-0644(05)81302-2

Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using SUMO. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582). IEEE. https://doi.org/10.1109/ITSC.2018.8569938

Nelson, E. J., & Bullock, D. (2000). Impact of emergency vehicle preemption on signalized corridor operation: An evaluation. *Transportation Research Record, 1727*(1), 1–11. https://doi.org/10.3141/1727-01

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Pemerintah Provinsi DKI Jakarta. (n.d.). *Data lampu lalu lintas terintegrasi Intelligent Transport System Area Traffic Control System (ITS ATCS)* [Data set]. Jakarta Open Data. https://data.jakarta.go.id/dataset/data-traffic-light-atcs-dki-jakarta

R Core Team. (2026). *R: A language and environment for statistical computing* (Version 4.6.1) [Computer software]. R Foundation for Statistical Computing. https://www.R-project.org/

Roy, S., Hunter, M., Saroj, A., & Guin, A. (2024). Development of emergency vehicle preemption strategies on smart corridors in a digital twin environment. In *Proceedings of the 17th ACM SIGSPATIAL International Workshop on Computational Transportation Science (IWCTS '24)*. ACM. https://doi.org/10.1145/3681772.3698213

Su, H., Zhong, Y. D., Chow, J. Y. J., Dey, B., & Jin, L. (2023). EMVLight: A multi-agent reinforcement learning framework for an emergency vehicle decentralized routing and traffic signal control system. *Transportation Research Part C: Emerging Technologies, 146*, Article 103955. https://doi.org/10.1016/j.trc.2022.103955

The Jakarta Post. (2024, January 16). Jakarta's global traffic ranking improves, but congestion worsens. *Asia News Network*. https://asianews.network/jakartas-global-traffic-ranking-improves-but-congestion-worsens/

Treiber, M., Hennecke, A., & Helbing, D. (2000). Congested traffic states in empirical observations and microscopic simulations. *Physical Review E, 62*(2), 1805–1824. https://doi.org/10.1103/PhysRevE.62.1805

UP.SPLL Dinas Perhubungan Provinsi DKI Jakarta. (n.d.). *ATCS Jakarta: Fungsi dan manfaat*. https://atcs-jakarta.com/

van Rossum, G., Warsaw, B., & Coghlan, A. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. Retrieved October 3, 2026, from https://peps.python.org/pep-0008/

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies, 36*, 177–195. https://doi.org/10.1016/j.trc.2013.08.014

Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Her Majesty's Stationery Office.

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019). *A survey on traffic signal control methods*. arXiv. https://doi.org/10.48550/arXiv.1904.08117

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data, 3*, Article 160018. https://doi.org/10.1038/sdata.2016.18

Appendix A. Worked numeric examples

- **Commit distance.** $v^2/(2b_{stop})$: 13.9 m/s → 32.2 m; 16.0 m/s → 42.7 m; 17.4 m/s (ambulance) → 50.5 m; 5 m/s → 4.2 m.
- **Box clearing time** of a just-committed vehicle: $t_{clear}(v) = v/6 + 20.5/v$: 5 m/s → 4.9 s; 11.1 m/s → 3.7 s; 16 m/s → 3.9 s; an ambulance (17.4 m/s, 6.0 m long, 22 m to clear) → 4.2 s.

- **Capacity.** $c_{formula} = 1631 \times 2 \times 18/96 = 612$ veh/h per two-lane approach; the fixed-time capacity the simulated junction actually realises, and against which the demand levels are calibrated, is $c = 549$ veh/h (90% of the formula value, Section 9).
- **Design demand.** $\lambda_{base} = 0.7 \times 549/1.18 = 326$ veh/h per entry; eight entries give about 2,605 vehicle requests per hour on the grid.
- **Webster (1958) at design load.** $d_1 = 96(1 - 18/96)^2/(2(1 - 0.7 \cdot 18/96)) = 36.5$ s; the simulated single-junction value at $x = 0.7$ was 43.8 s (total delay including entry queueing).
- **Storage.** $\lfloor 186/6.5 \rfloor = 28$ vehicles per internal lane; $\lfloor 143/6.5 \rfloor = 22$ per stub lane.
- **Teacher threshold.** An ambulance with three queued cars ahead of it needs $t_{clear} = 4 + l_{startup} + 3\,h_{sat}$, about 12 s with the measured values; the teacher asks for its green when the ETA falls to that number, the naive rule 20 s out regardless of the queue.

Appendix B. Implementation notes: where the generator refines the written design

The generator follows the project's design specification; these refinements were made while building it and are recorded so that the document describes what actually runs and how to read the data.

- Decision instants exist only during green, starting 5 s after onset, so the forced codes 1 (min green) and 3 (all-red) never occur; only 0 (none), 2 (max green) and 4 (starvation guard) appear.
- A vehicle that leaves a connector in a step is not integrated again by the edge loop of the same step (an earlier version advanced it an extra $v\,\Delta t$ at every junction crossing).
- `signal_ticks` carries `down_n_*` in addition to occupancy so that the label is recomputable from the row alone.
- An exit is "blocked" per edge when, on any of its lanes, the last vehicle is queued ($v < 2$ m/s) and its rear bumper leaves less than $l_{veh} + s_0 + 1 = 7.5$ m free at the lane start; a vehicle that is driving away does not block. A vehicle whose target lane is blocked treats the stop line as its leader only while it can still stop at b_{stop}; past that point it enters the box anyway and, if it halts inside, is counted in `n_box_blocked`. An approach with all three exits blocked gets pressure -10^9.
- The label picks ties by lowest index and, at max green, the highest-pressure other approach; the plant's forced switch picks the largest vehicle count in rotation order. The label is the request, not the applied phase.
- `y_mp_action` is computed on every row; `y_q_total_t30` is read at the first whole second $\geq t + 30$ (decision instants can fall on half seconds), is NaN on tail and cool-down rows, and is also NaN when the episode ended early through gridlock.
- The stop line is a zero-speed virtual leader at $L_e - 2$, so vehicles halt about 2 m before it and enter the box at L_e; the stop-or-commit evaluation runs at the decision instant before vehicles move.
- Turn anticipation uses a 120 m zone and half the driver's comfortable deceleration (the design specification said 60 m and the full b); see Section 3.4.
- A vehicle leaves the box when its front bumper passes the connector end; the clearance gate is therefore slightly optimistic by the time its rear needs.
- A hard brake is counted only while the vehicle is moving ($v > 0$).
- The 300 m ambulance horizon never binds on this grid (edges are 143 m and 186 m long), so `ev_present_a` is true from the moment an ambulance enters the approach edge; between the stop line and the box edge it is still reported, with `ev_dist_a = 0` and `ev_eta_a = 0`.
- `n_ambulances` counts ambulance requests over the whole episode; the `ev_mean_*` KPIs use ambulances requested in the main period. Ambulances store `alpha = 1.0`.
- At box entry a through or boundary-bound vehicle takes the exit lane with fewer vehicles, counting vehicles already on connectors to that lane; two vehicles released side by side that aim for the same

exit lane zipper-merge at the box entry, which shows up as hard brakes.

- `episodes.csv` has the extra health and liveness columns `n_entry_queued_end`, `n_invariant_violations`, `vehicle_steps`, `gridlock_junction`, `max_approach_red_s` (longest red faced by an occupied approach), `max_total_wait_s`, `n_forced_starvation`, `n_emergency_brakes` (IDM demanded more than the physical braking limit while moving), `n_commit_revoked`, `n_merge_holds`, `merge_hard_stops`, `max_head_wait_s`, `spillback_lock`, `n_spillback_locks`, `spillback_lock_t` and `spillback_lock_junction`. The car delay penalty is filled in by the generator from the paired no-ambulance run and is NaN when that run is skipped.
- All plants start at phase N, green, $t = 0$; under fixed time the four junctions run a synchronised cycle until all-red extensions desynchronise them.
- The sensor copy `obs_q_`$*$ is drawn on every decision row from the sensor stream; no controller reads it.
- The configuration saved with the dataset records the formula capacity, the measured capacity and their ratio next to the calibrated base rates, so the demand levels of any release can be re-derived.

Appendix C. Data dictionary of `signal_ticks`

column	dtype	unit	namespace	description
episode_id	int32	n/a	key	episode key (= seed - 100000)
junction_id	int8	n/a	key	junction key j*grid_x + i; NOT a feature
t	float32	s	time	decision instant; t = 0 is the start of warm-up
tick_idx	int32	n/a	time	running index of decision rows of this junction
is_warmup	bool	n/a	time	t < warmup_s; exclude from training and KPIs
is_cooldown	bool	n/a	time	t >= warmup_s + main_s (spawning off)
is_tail	bool	n/a	time	last 30 s of the main period (regression label NaN)
phase	int8	n/a	signal	approach that is green: 0 N, 1 E, 2 S, 3 W
time_in_phase_s	float32	s	signal	time since the current green began
max_green_remaining_s	float32	s	signal	max_green - time_in_phase_s
q_N	int16	veh	state	vehicles with v < 2 m/s on the approach edge from arm N (all lanes)
n_N	int16	veh	state	all vehicles on the approach edge from arm N
q_N_l0	int16	veh	state	queue on lane 0 (kerb lane); -1 when lanes_per_dir = 1
q_N_l1	int16	veh	state	queue on lane 1 (median lane); -1 when lanes_per_dir = 1
arr_N_30s	int16	veh	state	vehicles that entered the approach edge in (t-30, t]
q_E	int16	veh	state	vehicles with v < 2 m/s on the approach edge from arm E (all lanes)
n_E	int16	veh	state	all vehicles on the approach edge from arm E
q_E_l0	int16	veh	state	queue on lane 0 (kerb lane); -1 when lanes_per_dir = 1
q_E_l1	int16	veh	state	queue on lane 1 (median lane); -1 when lanes_per_dir = 1

column	dtype	unit	namespace	description
arr_E_30s	int16	veh	state	vehicles that entered the approach edge in (t-30, t]
q_S	int16	veh	state	vehicles with v < 2 m/s on the approach edge from arm S (all lanes)
n_S	int16	veh	state	all vehicles on the approach edge from arm S
q_S_l0	int16	veh	state	queue on lane 0 (kerb lane); -1 when lanes_per_dir = 1
q_S_l1	int16	veh	state	queue on lane 1 (median lane); -1 when lanes_per_dir = 1
arr_S_30s	int16	veh	state	vehicles that entered the approach edge in (t-30, t]
q_W	int16	veh	state	vehicles with v < 2 m/s on the approach edge from arm W (all lanes)
n_W	int16	veh	state	all vehicles on the approach edge from arm W
q_W_l0	int16	veh	state	queue on lane 0 (kerb lane); -1 when lanes_per_dir = 1
q_W_l1	int16	veh	state	queue on lane 1 (median lane); -1 when lanes_per_dir = 1
arr_W_30s	int16	veh	state	vehicles that entered the approach edge in (t-30, t]
down_n_N	int16	veh	downstream	vehicles on the outgoing edge toward N (stubs: real count)
down_occ_N	float32	n/a	downstream	down_n / storage of that edge (storage = per-lane storage x lanes)
down_blocked_N	bool	n/a	downstream	a queued vehicle leaves < l_veh+s0+1 m free at the start of some lane of that edge
down_n_E	int16	veh	downstream	vehicles on the outgoing edge toward E (stubs: real count)
down_occ_E	float32	n/a	downstream	down_n / storage of that edge (storage = per-lane storage x lanes)
down_blocked_E	bool	n/a	downstream	a queued vehicle leaves < l_veh+s0+1 m free at the start of some lane of that edge
down_n_S	int16	veh	downstream	vehicles on the outgoing edge toward S (stubs: real count)
down_occ_S	float32	n/a	downstream	down_n / storage of that edge (storage = per-lane storage x lanes)
down_blocked_S	bool	n/a	downstream	a queued vehicle leaves < l_veh+s0+1 m free at the start of some lane of that edge
down_n_W	int16	veh	downstream	vehicles on the outgoing edge toward W (stubs: real count)
down_occ_W	float32	n/a	downstream	down_n / storage of that edge (storage = per-lane storage x lanes)
down_blocked_W	bool	n/a	downstream	a queued vehicle leaves < l_veh+s0+1 m free at the start of some lane of that edge

column	dtype	unit	namespace	description
nb_N_exists	bool	n/a	neighbour	there is a neighbour junction in direction N
nb_N_phase	int8	n/a	neighbour	neighbour's phase (green or most recently green); -1 if none
nb_N_green_toward_me	bool	n/a	neighbour	neighbour is GREEN for an approach that feeds the link toward this junction
nb_E_exists	bool	n/a	neighbour	there is a neighbour junction in direction E
nb_E_phase	int8	n/a	neighbour	neighbour's phase (green or most recently green); -1 if none
nb_E_green_toward_me	bool	n/a	neighbour	neighbour is GREEN for an approach that feeds the link toward this junction
nb_S_exists	bool	n/a	neighbour	there is a neighbour junction in direction S
nb_S_phase	int8	n/a	neighbour	neighbour's phase (green or most recently green); -1 if none
nb_S_green_toward_me	bool	n/a	neighbour	neighbour is GREEN for an approach that feeds the link toward this junction
nb_W_exists	bool	n/a	neighbour	there is a neighbour junction in direction W
nb_W_phase	int8	n/a	neighbour	neighbour's phase (green or most recently green); -1 if none
nb_W_green_toward_me	bool	n/a	neighbour	neighbour is GREEN for an approach that feeds the link toward this junction
obs_q_N	int16	veh	sensor	Binomial(q_a, 1-p_miss) + Poisson(p_miss/2), p_miss = sensor_noise_level; = q_a at 0
obs_q_E	int16	veh	sensor	Binomial(q_a, 1-p_miss) + Poisson(p_miss/2), p_miss = sensor_noise_level; = q_a at 0
obs_q_S	int16	veh	sensor	Binomial(q_a, 1-p_miss) + Poisson(p_miss/2), p_miss = sensor_noise_level; = q_a at 0
obs_q_W	int16	veh	sensor	Binomial(q_a, 1-p_miss) + Poisson(p_miss/2), p_miss = sensor_noise_level; = q_a at 0
ev_present_N	bool	n/a	ev	an ambulance is on the approach edge from arm N within ev_horizon_m of the stop line (GPS channel, no noise)
ev_dist_N	float32	m	ev	distance of the nearest ambulance to the stop line; -1 if none
ev_eta_N	float32	s	ev	ev_dist / max(v_ambulance, 3 m/s); -1 if none
ev_q_ahead_N	int16	veh	ev	queued vehicles in the ambulance's lane ahead of it; -1 if none (input of the priority_mp teacher)
ev_present_E	bool	n/a	ev	an ambulance is on the approach edge from arm E within ev_horizon_m of the stop line (GPS channel, no noise)

column	dtype	unit	namespace	description
ev_dist_E	float32	m	ev	distance of the nearest ambulance to the stop line; -1 if none
ev_eta_E	float32	s	ev	ev_dist / max(v_ambulance, 3 m/s); -1 if none
ev_q_ahead_E	int16	veh	ev	queued vehicles in the ambulance's lane ahead of it; -1 if none (input of the priority_mp teacher)
ev_present_S	bool	n/a	ev	an ambulance is on the approach edge from arm S within ev_horizon_m of the stop line (GPS channel, no noise)
ev_dist_S	float32	m	ev	distance of the nearest ambulance to the stop line; -1 if none
ev_eta_S	float32	s	ev	ev_dist / max(v_ambulance, 3 m/s); -1 if none
ev_q_ahead_S	int16	veh	ev	queued vehicles in the ambulance's lane ahead of it; -1 if none (input of the priority_mp teacher)
ev_present_W	bool	n/a	ev	an ambulance is on the approach edge from arm W within ev_horizon_m of the stop line (GPS channel, no noise)
ev_dist_W	float32	m	ev	distance of the nearest ambulance to the stop line; -1 if none
ev_eta_W	float32	s	ev	ev_dist / max(v_ambulance, 3 m/s); -1 if none
ev_q_ahead_W	int16	veh	ev	queued vehicles in the ambulance's lane ahead of it; -1 if none (input of the priority_mp teacher)
action_requested	int8	n/a	action	phase requested by the driving controller
action_applied	int8	n/a	action	phase in force after the plant's filter (the pending phase when a switch was accepted)
forced	int8	n/a	action	0 none, 1 min_green, 2 max_green, 3 all_red, 4 starvation guard (an occupied approach had no green for starvation_s)
switched	bool	n/a	action	a switch (all-red) started at this instant
y_mp_action	int8	n/a	label	request of the priority-aware teacher priority_mp recomputed from this row (label for imitation): the ambulance's approach when ev_eta_a <= allred_min + startup lost time + ev_q_ahead_a x saturation headway, otherwise the max-pressure request; identical to the max-pressure request on every row without an ambulance
y_q_total_t30	float32	veh	label	q_total of this junction at the first whole second >= t+30 (junction_1hz)